

**Universitatea  
Transilvania  
din Braşov**

**FACULTATEA DE INGINERIE ELECTRICĂ  
ŞI ŞTIINŢA CALCULATOARELOR**

# **PROIECT DE DIPLOMĂ**

Conducători ştiinţifici:

Conf. Dr. Ing. DANCIU Gabriel Mihail

Ş. L. Dr. Ing. KERTÉSZ Csaba Zoltán

Absolvent:

CILICHIDREANU Andi-Marian

**Braşov, 2026**

Departamentul de Electronică și Calculatoare  
Programul de studii: Calculatoare

**CILICHIDREANU Andi-Marian**

# Sistem de monitorizare al unei ferme de ciuperci

## **Conducători științifici:**

Conf. Dr. Ing. DANCIU Gabriel Mihail

Ș. L. Dr. Ing. KERTÉSZ Csaba Zoltán

Brașov, 2026

# Cuprins

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Lista de figuri, tabele și coduri sursă . . . . .                                   | 1  |
| Lista de acronime . . . . .                                                         | 6  |
| 1 Introducere . . . . .                                                             | 8  |
| 1.1 Contextul și importanța temei . . . . .                                         | 8  |
| 1.2 Analiza soluțiilor existente și originalitatea proiectului . . . . .            | 9  |
| 1.2.1 Analiza pieței și a produselor comerciale concurente . . . . .                | 9  |
| 1.2.2 Elemente de originalitate ale soluției propuse . . . . .                      | 10 |
| 1.3 Scopul și obiectivele lucrării . . . . .                                        | 11 |
| 1.4 Grupul țintă și impactul estimat . . . . .                                      | 12 |
| 1.4.1 Grupul țintă . . . . .                                                        | 12 |
| 1.4.2 Impactul economic . . . . .                                                   | 12 |
| 1.4.3 Impactul ecologic și eficiența resurselor . . . . .                           | 13 |
| 1.5 Delimitările și limitările sistemului . . . . .                                 | 13 |
| 1.5.1 Delimitări arhitecturale și de scalabilitate fizică . . . . .                 | 13 |
| 1.5.2 Delimitări funcționale . . . . .                                              | 14 |
| 1.5.3 Limitări de infrastructură . . . . .                                          | 14 |
| 1.6 Structurarea lucrării pe capitole . . . . .                                     | 14 |
| 2 Fundamente teoretice . . . . .                                                    | 16 |
| 2.1 Edge Computing în monitorizarea distribuită . . . . .                           | 16 |
| 2.2 Fiabilitatea componentelor electronice în medii cu umiditate ridicată . . . . . | 16 |
| 2.3 Protocoale de comunicație folosite și achiziția digitală de date . . . . .      | 17 |
| 2.3.1 Conversia Analog-Digital (ADC) . . . . .                                      | 17 |
| 2.3.2 Protocolul I <sup>2</sup> C . . . . .                                         | 18 |
| 2.3.3 Magistrala 1-Wire . . . . .                                                   | 18 |
| 2.4 Modele de stocare și mesagerie în timp real . . . . .                           | 19 |
| 2.4.1 Avantajele bazelor de date NoSQL . . . . .                                    | 19 |
| 2.4.2 Tehnici de sincronizare în timp real a datelor . . . . .                      | 20 |
| 2.5 Securitatea aplicațiilor mobile și modelul Zero-Trust . . . . .                 | 21 |
| 2.6 Tehnologii, platforme și medii de dezvoltare utilizate . . . . .                | 22 |
| 2.6.1 Single-Board Computer (SBC) . . . . .                                         | 22 |
| 2.6.2 Limbajul de programare Python . . . . .                                       | 22 |
| 2.6.3 Limbajul de programare TypeScript . . . . .                                   | 23 |
| 2.6.4 React Native și platforma Expo . . . . .                                      | 24 |
| 2.6.5 Firebase . . . . .                                                            | 24 |
| 2.6.6 Ngrok . . . . .                                                               | 25 |
| 2.6.7 GitHub . . . . .                                                              | 25 |
| 3 Proiectarea și implementarea sistemului . . . . .                                 | 27 |
| 3.1 Arhitectura generală a sistemului . . . . .                                     | 27 |
| 3.2 Proiectarea hardware și alegerea componentelor . . . . .                        | 28 |
| 3.2.1 Alegerea unității centrale de procesare . . . . .                             | 28 |

|       |                                                                                                   |    |
|-------|---------------------------------------------------------------------------------------------------|----|
| 3.2.2 | Integrarea convertorului analog-digital . . . . .                                                 | 29 |
| 3.2.3 | Motivarea alegerii senzorilor utilizați . . . . .                                                 | 30 |
| 3.2.4 | Integrarea interfeței vizuale locale . . . . .                                                    | 32 |
| 3.2.5 | Integrarea dispozitivului de monitorizare vizuală . . . . .                                       | 33 |
| 3.2.6 | Realizarea circuitului fizic . . . . .                                                            | 33 |
| 3.3   | Dezvoltarea software-ului de achiziție și control . . . . .                                       | 35 |
| 3.3.1 | Arhitectura aplicației locale și mediul de dezvoltare . . . . .                                   | 36 |
| 3.3.2 | Achiziția datelor și prelucrarea matematică a semnalelor . . . . .                                | 37 |
| 3.3.3 | Sistemul de captură vizuală . . . . .                                                             | 41 |
| 3.3.4 | Schema logică a fluxului principal de execuție . . . . .                                          | 42 |
| 3.4   | Arhitectura platformei Cloud . . . . .                                                            | 42 |
| 3.4.1 | Dezvoltarea modului de comunicare cu baza de date . . . . .                                       | 44 |
| 3.4.2 | Gestionarea fluxului de rețea . . . . .                                                           | 47 |
| 3.4.3 | Sistemul de alerte și integrarea notificărilor push . . . . .                                     | 49 |
| 3.5   | Dezvoltarea aplicației mobile pentru monitorizare și control . . . . .                            | 51 |
| 3.5.1 | Arhitectura aplicației client și mediul de dezvoltare . . . . .                                   | 52 |
| 3.5.2 | Interfața de monitorizare în timp real și evaluarea stării sistemului . . . . .                   | 56 |
| 3.5.3 | Vizualizarea analitică a datelor istorice și modulul de export . . . . .                          | 57 |
| 3.5.4 | Configurarea de la distanță a sistemului și gestionarea preferințelor<br>utilizatorului . . . . . | 58 |
| 3.5.5 | Sistemul de monitorizare vizuală în timp real . . . . .                                           | 60 |
| 3.5.6 | Sinteza navigării și experiența utilizatorului . . . . .                                          | 61 |
| 4     | Concluzii . . . . .                                                                               | 65 |
| 4.1   | Îndeplinirea obiectivelor și rezultate obținute . . . . .                                         | 65 |
| 4.2   | Contribuții personale . . . . .                                                                   | 65 |
| 4.3   | Evaluarea metricilor de performanță . . . . .                                                     | 66 |
| 4.4   | Analiza tehnico-economică și evaluarea soluțiilor de pe piață . . . . .                           | 68 |
| 4.5   | Direcții viitoare de cercetare și dezvoltare . . . . .                                            | 70 |
|       | Bibliografie . . . . .                                                                            | 76 |
|       | Rezumat . . . . .                                                                                 | 77 |
|       | Abstract . . . . .                                                                                | 78 |

## FIGURI

|    |                                                                                                                                     |    |
|----|-------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Exemplu de conectare a mai multor dispozitive I <sup>2</sup> C pe aceeași magistrală [12]                                           | 18 |
| 2  | Configurația hardware a unui dispozitiv 1-Wire [14]                                                                                 | 19 |
| 3  | Diferența dintre metoda Polling și WebSocket [19]                                                                                   | 20 |
| 4  | Modelul de mesagerie Publish-Subscribe [20]                                                                                         | 21 |
| 5  | Arhitectura generală a sistemului                                                                                                   | 28 |
| 6  | Pinout Raspberry Pi 5 [45]                                                                                                          | 29 |
| 7  | Convertorul Analog-Digital ADS1115 [47]                                                                                             | 30 |
| 8  | Ecranul OLED SSD1306 [54]                                                                                                           | 32 |
| 9  | Schema electronică a sistemului                                                                                                     | 34 |
| 10 | Așezarea componentelor pe placa perforată                                                                                           | 35 |
| 11 | Schema logică a fluxului de execuție                                                                                                | 43 |
| 12 | Arhitectura aplicației mobile                                                                                                       | 54 |
| 13 | Schema logică a algoritmului de reducere și generare controlată a etichetelor axei X                                                | 59 |
| 14 | Schema logică a fluxului de navigare a utilizatorului - User Flowchart                                                              | 61 |
| 15 | Ecranul de autentificare                                                                                                            | 62 |
| 16 | Interfața aplicației: modulul principal, monitorizarea, analiza istorică și configurarea                                            | 63 |
| 17 | Demonstrarea separării fluxului de scriere pe microSD prin monitorizarea memoriei RAM și a proceselor de scriere pe disc            | 66 |
| 18 | Monitorizarea consumului de resurse pentru procesele concurente ale sistemului local                                                | 67 |
| 19 | Statisticile de trafic pentru interfața wireless wlan0                                                                              | 67 |
| 20 | Monitorizarea resurselor de calcul și a temperaturii interne a unității centrale în timpul funcționării sistemului de răcire activă | 69 |

## TABELE

|   |                                                                                  |    |
|---|----------------------------------------------------------------------------------|----|
| 1 | Costul estimativ al componentelor hardware utilizate în dezvoltarea prototipului | 69 |
|---|----------------------------------------------------------------------------------|----|

## CODURI SURSĂ

|   |                                                                                |    |
|---|--------------------------------------------------------------------------------|----|
| 1 | Implementarea arhitecturii concurente și mecanismul de evitare a overflow-ului | 37 |
| 2 | Implementarea limitării superioare/inferioare și maparea liniară a umidității  | 38 |

|    |                                                                                                   |    |
|----|---------------------------------------------------------------------------------------------------|----|
| 3  | Implementarea algoritmului de calcul exponențial pentru concentrația de CO <sub>2</sub> . . . . . | 40 |
| 4  | Transpunerea ecuației Tetens în Python pentru determinarea VPD-ului . . . . .                     | 41 |
| 5  | Implementarea mecanismului asincron de scriere pentru date în timp real și istoric .              | 45 |
| 6  | Implementarea _settings_listener pentru actualizarea pragurilor în timp real . . . . .            | 45 |
| 7  | Implementarea funcției de trimitere a alertelor la interval de 15 minute . . . . .                | 46 |
| 8  | Implementarea thread-ului secundar pentru procesarea asincronă a pachetelor de date               | 48 |
| 9  | Algoritmul de calcul al buffer-elor și evaluarea valorilor . . . . .                              | 49 |
| 10 | Integrarea API-ului Expo pentru transmiterea cererilor HTTP de tip Push Notification              | 50 |
| 11 | Implementarea securității biometrice la reactivarea aplicației din fundal . . . . .               | 52 |
| 12 | Implementarea mecanismului matematic pentru evaluarea stării conexiunii IoT . . . . .             | 56 |

ADC - Analog-to-Digital Converter;  
AO - Analog Output;  
API - Application Programming Interface;  
BaaS - Backend-as-a-Service;  
CAD - Computer-Aided Design;  
CFF - Conductive Filament Formation;  
CPU - Central Processing Unit;  
CSV - Comma-Separated Values;  
ES - Embedded Systems;  
GND - Ground;  
GPIO - General-Purpose Input/Output;  
HTTP - Hypertext Transfer Protocol;  
HTTPS - Hypertext Transfer Protocol Secure;  
HVAC - Heating, Ventilation and Air Conditioning;  
I/O - Input/Output;  
I<sup>2</sup>C - Inter-Integrated Circuit;  
IDE - Integrated Development Environment;  
IoT - Internet of Things;  
IP - Internet Protocol;  
JPG - Joint Photographic Experts Group;  
JSON - JavaScript Object Notation;  
JST - Japan Solderless Terminal;  
kPa - kilopascali;  
LCD - Liquid Crystal Display;  
LPDDR - Low-Power Double Data Rate;  
MB - Megabyte;  
ML - Machine Learning;  
MQTT - Message Queuing Telemetry Transport;  
NTC - Negative Temperature Coefficient;  
OLED - Organic Light-Emitting Diode;  
OOP - Object-Oriented Programming;  
OS - Operating System;  
PCB - Printed Circuit Board;  
PLC - Programmable Logic Controller;  
PPM - Parts Per Million;  
PWM - Pulse Width Modulation;  
RAM - Random Access Memory;  
ROM - Read Only Memory;  
RSS - Resident Set Size;  
SBC - Single-Board Computer;  
SCL - Serial Clock;  
SD - Secure Digital;  
SDA - Serial Data Line;  
SDK - Software Development Kit;  
SDRAM - Synchronous Dynamic Random Access Memory;  
SoC - System on a Chip;

SQL - Structured Query Language;  
SVP - Saturated Vapour Pressure;  
TCP - Transmission Control Protocol;  
UI - User Interface;  
UPS - Uninterruptible Power Supply;  
URL - Uniform Resource Locator;  
USB - Universal Serial Bus;  
UVC - USB Video Class;  
VPD - Vapor Pressure Deficit;



# 1 Introducere

Cultivarea ciupercilor reprezintă una dintre cele mai delicate și complexe ramuri ale agriculturii moderne. Eficiența acestora depinde direct de controlul precis, automatizat și în timp real al factorilor de mediu. Spre deosebire de plantele verzi, care utilizează fotosinteza și au o rezistență mai ridicată la variațiile climatice, ciupercile sunt organisme heterotrofe. Ele respiră într-o manieră similară organismelor animale, absorbind oxigen și eliminând dioxid de carbon, în funcție de etapa de dezvoltare în care se află.

Ciupercile nu dispun de structuri celulare pentru retenția apei sau cuticule protectoare împotriva deshidratării, ceea ce le face extrem de sensibile la cele mai mici fluctuații ale umidității relative din aer și din sol. În acest context, ingineria sistemelor computerizate și conceptele moderne de IoT devin esențiale. Acestea transformă monitorizarea agricolă dintr-un proces bazat pe intervenția umană continuă, într-o știință exactă, condusă de bucle de control în circuit închis.

## 1.1 Contextul și importanța temei

Importanța monitorizării stricte a unui microclimat fungic constă în dinamica biologică particulară a culturilor micologice. Parametri precum temperatura aerului, umiditatea relativă, concentrația de  $\text{CO}_2$ , intensitatea luminoasă și umiditatea solului interacționează în mod colaborativ, definind rata de evaporare la nivelul pălăriei ciupercii și stimulând schimbul de gaze. De exemplu, în faza de fructificare, o concentrație excesivă de  $\text{CO}_2$  (peste 1000-1200 PPM) duce inevitabil la deformarea severă a corpurilor de fructificare, prin alungirea excesivă a piciorului și subdezvoltarea pălăriei, scăzând astfel valoarea comercială a recoltei sau compromițând-o în totalitate [1]. În mod similar, menținerea unei umidități a aerului în intervalul critic de 80-90% este obligatorie pentru unele specii de ciuperci [2]. Dacă nivelul de umiditate scade sub acest prag precizat, ciupercile pot să se usuce, iar depășirea acestui prag, în absența unei ventilații adecvate, favorizează apariția mucegaiului și a altor microorganisme periculoase [2].

Proiectul "Sistem de monitorizare al unei ferme de ciuperci" a fost conceput ca răspuns direct la această provocare tehnologică și biologică. Automatizarea microclimatului reduce dependența de factorul uman și elimină erorile asociate măsurărilor manuale periodice. Prin utilizarea microcalculatorului Raspberry Pi 5, sistemul propus nu se limitează doar la simpla achiziție și transmitere a datelor, ci acționează ca un nod computațional de tip Edge Computing. Acest lucru înseamnă că deciziile critice de control, precum generarea alertelor locale sunt procesate direct la nivelul incintei de cultivare, oferind o stabilitate, o toleranță la erori și o viteză de reacție mult superioare sistemelor

dependente exclusiv de conexiunea la internet sau de platforme Cloud centralizate.

## 1.2 Analiza soluțiilor existente și originalitatea proiectului

### 1.2.1 Analiza pieței și a produselor comerciale concurente

Pentru a fundamenta necesitatea dezvoltării acestui sistem, a fost realizat un studiu al pieței asupra produselor comerciale deja existente, care vizează atât controlul microclimatului fungic, cât și a anumitor parametri cu un impact major asupra creșterii culturii fungice. Oferta actuală se împarte în mai multe categorii tehnologice, fiecare prezentând limitări specifice în raport cu nevoile unei ferme de mici dimensiuni flexibile:

- **Ferme inteligente de tip "Smart Appliance" (Shrooly):** Reprezintă o categorie de dispozitive complet automatizate destinate exclusiv consumatorilor casnici. Deși oferă o experiență de utilizare excelentă, integrând senzori, ventilație, iluminat și o aplicație mobilă dedicată, ele prezintă limitări critice de scalabilitate. Arhitectura este una închisă și adesea restrictivă, obligând utilizatorul să folosească substraturi specifice. Mai mult, volumul de cultivare este strict limitat la dimensiunile aparatului, făcând imposibilă extinderea sistemului pentru o incintă de capacitate medie [3].
- **Controllere de mediu independente (Inkbird):** Acestea sunt dispozitive de tip "plug-and-play" foarte populare, dotate cu un senzor și prize de acționare. Deși sunt fiabile și accesibile, acestea generează seturi de date fragmentate, fără posibilitatea integrării într-un sistem centralizat. Utilizatorul nu are o imagine de ansamblu corelată, iar funcționalitățile aplicațiilor mobile sunt simple, lipsite de capabilități de Edge Computing sau alerte predictive complexe [4].
- **Echipamente agricole semi-industriale (TrolMaster):** Reprezintă sisteme modulare de înaltă precizie, capabile să controleze zeci de parametri simultan. Principalul dezavantaj al acestora pentru grupul țintă îl reprezintă prețul. Costul unui modul central, care poate să fie extins cu ajutorul unor senzori dedicați, depășește bugetul de pornire al unei ferme pentru micii cultivatori [5].
- **Sisteme IoT comerciale de tip Smart Home:** Deși ecosisteme precum Samsung SmartThings sau Google Nest oferă arhitecturi SoC compacte și accesibile, ele prezintă limitări critice într-o fermă de ciuperci. Hardware-ul folosit în platformele precizate se degradează rapid la expunerea pe un termen lung la o umiditate relativă de 80-90%. De asemenea, literatura din domeniul *Smart Agriculture* pune accentul asupra vulnerabilității majore a acestor sisteme,

mai exact dependența exclusivă de procesarea în Cloud. Lucrarea [6] prezintă faptul că latența specifică rețelei și întreruperile de conectivitate blochează complet luarea deciziilor în timp real și opresc bucla de automatizare. Pentru a elimina aceste riscuri, standardul industrial actual a migrat către descentralizarea deciziilor direct la sursa de date, impunând utilizarea arhitecturilor de tip Edge Computing [7].

Prin contrast, sistemul propus în această lucrare preia avantajele software și gradul înalt de automatizare al dispozitivelor prezentate, dar elimină limitările de volum printr-o arhitectură hardware deschisă și modulară, bazată pe Raspberry Pi. Acest fapt permite sistemului să fie montat pe orice tip de incintă, menținând totodată costurile de dezvoltare la nivelul echipamentelor de bază și garantând monitorizarea integrală a datelor printr-o infrastructură Cloud privată.

### 1.2.2 Elemente de originalitate ale soluției propuse

Sistemul dezvoltat în cadrul acestei lucrări se poziționează la intersecția dintre accesibilitatea sistemelor IoT și fiabilitatea arhitecturilor industriale, aducând o serie de inovații tehnice precum:

- **Arhitectura de izolare fizică și protecție ambientală a nodului central:** În mediile cu umiditate extremă, electronica de calcul este expusă unui risc major de condens, oxidare rapidă și scurtcircuit. Soluția prezentată în acest proiect constă în proiectarea unei bariere fizice complete prin montarea nodului central Raspberry Pi 5 exclusiv pe exteriorul incintei de fructificare. Interfațarea cu microclimatul intern se realizează strict prin magistrale de date ale senzorilor.
- **Coordonarea Edge-Cloud:** Toți algoritmi de decizie rulează exclusiv local pe Raspberry Pi 5, în timp ce platforma Cloud (Firebase) este utilizată strict pentru transmiterea și stocarea datelor. În cazul unei deconectări de la rețea, algoritmi de stabilizare și decizie continuă să ruleze pe Raspberry Pi 5, parametrii de mediu fiind în continuare preluați de către senzori și afișați local pe ecranul OLED.
- **Algoritmul de alertare dinamică:** Spre deosebire de sistemele clasice cu praguri binare (pornit/oprit), software-ul calculează în timp real zone de buffer dinamice (15% înainte de atingerea limitelor critice), clasificând stările în **Info**, **Warning** și **Critical**, optimizând astfel consumul de lățime de bandă și livrarea notificărilor push.
- **Model de securitate Zero-Trust la nivel de aplicație mobilă:** Interfața cu utilizatorul este realizată printr-o aplicație cross-platform construită în React Native și Expo. Aceasta integrează o arhitectură software proprie de tip Zero-Trust, limitată la nivelul dispozitivului mobil. Aceasta

validează fiecare sesiune prin autentificare biometrică locală și folosește un algoritm dedicat pentru detectarea deconectării sistemului fizic, determinat prin calculul diferenței dintre ceasul telefonului și timestamp-ul transmis de Raspberry Pi 5 în Firebase.

### 1.3 Scopul și obiectivele lucrării

Scopul principal al lucrării de licență constă în proiectarea, implementarea, calibrarea și validarea practică a unui sistem complet hardware și software, destinat monitorizării automate și optimizării microclimatului dintr-o fermă de ciuperci, utilizând tehnologii **Edge Computing** și **Cloud Computing** de înaltă performanță.

Pentru atingerea acestui scop, au fost definite și îndeplinite următoarele obiective:

- **Proiectarea și realizarea arhitecturii hardware:** Integrarea pe o placă perforată a unui ansamblu de senzori format din senzorul digital de temperatură DS18B20, senzorul de umiditate a aerului AM2302, senzorul de lumină BH1750 și senzorii analogici (gaz MQ-135 și umiditate sol HW-101) prin intermediul convertorului analog-digital extern ADS1115.
- **Dezvoltarea driverelor software:** Scrierea de la zero și optimizarea claselor de abstractizare în Python 3 pentru gestionarea senzorilor. Această etapă presupune implementarea algoritmilor de filtrare numerică a zgomotului de fond prin tehnici de eșantionare multiplă, compensarea matematică a divizoarelor de tensiune și calibrarea (precum determinarea constantei  $R_o$  pentru senzorul MQ-135) pentru a asigura acuratețea conversiilor analog-digitale.
- **Integrarea monitorizării hardware pentru nodul Edge:** Dezvoltarea unui modul software pentru monitorizarea în timp real a stării de funcționare a Raspberry Pi 5. Aceasta implică citirea temperaturii procesorului direct din sistemul de operare și transmiterea acesteia în Cloud, oferind utilizatorului un control vizual asupra stării de suprasolicitare termică a sistemului de achiziție.
- **Implementarea interfeței vizuale locale:** Integrarea unui display OLED SSD1306 utilizând bibliotecă grafică Luma pentru redarea în timp real a parametrilor direct la nivelul sistemului fizic.
- **Dezvoltarea sistemului de monitorizare vizuală în timp real:** Integrarea unei camere web prin interfață USB și implementarea unui fir de execuție independent, responsabil cu eșantionarea periodică a cadrelor, optimizarea latențelor prin utilizarea memoriei RAM pentru stocarea temporară a imaginilor și expunerea securizată a fluxului pe internet prin servicii de tunelare, precum Ngrok.

- **Configurarea infrastructurii Cloud:** Structurarea bazei de date Firebase Realtime Database pentru asigurarea unei sincronizări bidirecționale cu latență minimă între Raspberry Pi 5 și aplicația mobilă.
- **Dezvoltarea aplicației mobile:** Dezvoltarea unei interfețe grafice moderne și performante în React Native, capabilă să preia fluxurile de date în timp real, să afișeze grafice istorice cu posibilitate de export CSV și să permită configurarea de la distanță a pragurilor necesare de operare.

## 1.4 Grupul țintă și impactul estimat

Dezvoltarea sistemelor de tip IoT capătă o valoare reală doar în momentul în care tehnologia se adresează unei necesități concrete din piață. Astfel, soluția propusă a fost proiectată având în vedere un grup țintă bine definit, urmărind să genereze un impact măsurabil pe mai multe arii.

### 1.4.1 Grupul țintă

Sistemul se adresează în mod direct micilor fermieri, producătorilor de ciuperci (precum cultivatorii de *Pleurotus*, *Champignon* sau specii medicinale precum *Shiitake*) și antreprenorilor din sectorul agriculturii urbane (*urban agriculture*). Acest segment de utilizatori are nevoie de un control riguros al microclimatului pentru a obține recolte rentabile, dar nu dispune de puterea financiară necesară pentru achiziționarea unor echipamente industriale de tip PLC, menționate în analiza pieței. În plan secundar, datorită arhitecturii deschise și a monitorizării datelor în platforme Cloud, sistemul poate servi drept instrument pentru instituțiile de cercetare biologică, laboratoare universitare sau licee cu profil agricol, facilitând studiul evoluției ciupercilor în medii perfect controlate.

### 1.4.2 Impactul economic

Din punct de vedere economic, implementarea acestei arhitecturi hibride contribuie la deschiderea accesului la agricultura de precizie către o populație mult mai mare. Prin înlocuirea modulelor industriale scumpe cu microcalculatoare accesibile și prin dezvoltarea software-ului de la zero, bariera de intrare financiară pentru automatizarea unei ferme de dimensiuni mici este redusă substanțial. Mai mult, algoritmul de alertare dinamică și reacția automată a sistemului la schimbările parametrilor de mediu reduc semnificativ rata de compromitere a culturilor, asigurând un randament de producție constant și fără erori.

### 1.4.3 Impactul ecologic și eficiența resurselor

Din punct de vedere tehnic și ecologic, proiectul propune o tranziție de la funcționarea continuă a echipamentelor, la o funcționare orientată pe eveniment (*event-driven*). Utilizarea buclei de control bazate pe histerezis previne suprasolicitarea actuatorilor. Astfel, sistemele de răcire sunt acționate strict în intervalele de timp necesare corectării parametrilor (de exemplu, când concentrația de CO<sub>2</sub> se apropie de limita critică de 1000 PPM). Această metodă optimizează considerabil consumul de energie electrică, aliniind funcționarea fermei la principiile sustenabilității și ale agriculturii inteligente (*Smart Agriculture*).

## 1.5 Delimitările și limitările sistemului

Orice proiect ce vizează ingineria sistemelor necesită o definire clară a limitelor arhitecturale și funcționale, pentru a stabili așteptări realiste privind scalabilitatea și aplicabilitatea sa practică. Deși sistemul dezvoltat rezolvă problema monitorizării și a controlului automatizat, acesta prezintă o serie de delimitări asumate încă din faza de proiectare, precum și limitări tehnice specifice stadiului de prototip.

### 1.5.1 Delimitări arhitecturale și de scalabilitate fizică

Sistemul curent este conceput ca un prototip funcțional destinat monitorizării și controlului mediului în incinte de dimensiuni reduse. În configurația actuală, disiparea termică este gestionată de modulul Active Cooler al Raspberry Pi 5, acesta fiind folosit strict pentru menținerea temperaturii optime a procesorului în regim de funcționare continuă.

Deoarece ferma nu utilizează un sistem de climatizare integrat, controlul microclimatului este limitat la capacitățile pasive ale incintei. În forma sa hardware prezentă, sistemul nu este destinat utilizării directe în hale industriale de mari dimensiuni, întrucât reglarea precisă a parametrilor ambiențiali într-un volum mare ar necesita actuatori de putere (precum sisteme HVAC sau ventilatoare industriale) care depășesc capacitatea de curent a pinilor GPIO ai microcalculatorului.

Totuși, arhitectura software a fost proiectată cu o abordare modulară, permițând scalarea sistemului prin integrarea unor actuatori externi de putere mare. Această extindere poate fi realizată ușor prin intermediul unor relee care să izoleze circuitele de comandă de cele de putere, permițând astfel aplicației să controleze echipamente de climatizare industriale.

### 1.5.2 Delimitări funcționale

Scopul exclusiv al sistemului îl reprezintă gestionarea parametrilor de mediu (temperatură, umiditate, nivel de CO<sub>2</sub> și luminozitate). Prin urmare, proiectul nu integrează module de inteligență artificială sau viziune computerizată (Computer Vision) prin detectarea automată a bolilor fungice, a mucegaiurilor sau pentru evaluarea stadiului optim de recoltare a corpurilor de fructificare. Astfel, deciziile legate de biologia avansată a recoltei rămân o sarcină exclusiv umană. Totuși, sistemul dispune de o cameră web, ce îi permite cultivatorului să monitorizeze vizual și de la distanță starea culturii sale.

### 1.5.3 Limitări de infrastructură

O limitare clasică a arhitecturii hardware actuale o reprezintă dependența de o rețea electrică stabilă. Prototipul nu integrează un modul de alimentare neîntreruptibilă (UPS) sau surse de energie regenerabilă. Din acest motiv, o întrerupere a alimentării cu energie electrică va opri complet atât monitorizarea, cât și bucla de control local. În ceea ce privește conectivitatea, deși arhitectura de tip Edge Computing garantează menținerea parametrilor de climă și controlul actuatorilor chiar și în absența internetului, funcțiile superioare ale sistemului, precum livrarea alertelor push în timp real pe dispozitivul mobil, salvarea istoricului și modificarea pragurilor de la distanță, depind în mod obligatoriu de o conexiune Wi-Fi activă pentru sincronizarea cu Firebase.

## 1.6 Structurarea lucrării pe capitole

Lucrarea este structurată logic în patru capitole principale, realizate pentru a ghida cititorul prin parcursul întregului proces de dezvoltare inginerescă, de la fundamente teoretice până la analiza rezultatelor experimentale:

- **Capitolul 1 - Introducere:** Argumentează importanța temei, prezintă contextul biologic și tehnologic al culturii ciupercilor, realizează o analiză comparativă a stadiului actual al dezvoltării în domeniu pentru a scoate în evidență originalitatea soluției propuse și, nu în ultimul rând, definește scopul și obiectivele proiectului.
- **Capitolul 2 - Fundamente teoretice:** Prezintă sinteza conceptelor științifice și tehnologice necesare pentru descrierea arhitecturii sistemului. Sunt detaliate principiile fizice de conectare a senzorilor, modelul de calcul Edge Computing specific platformelor hardware hibride, precum și modelele de software utilizate în rețeaua IoT.

- **Capitolul 3 - Proiectarea și implementarea sistemului:** Detaliază transpunerea conceptelor teoretice în realizarea practică a prototipului. Acest capitol descrie arhitectura hardware și motivarea alegerii componentelor fizice, dezvoltarea software-ului de achiziție local pe unitatea Raspberry Pi 5, configurarea infrastructurii Cloud pentru gestionarea asincronă a datelor și proiectarea aplicației mobile cross-platform pentru monitorizarea în timp real și controlul de la distanță al fermei.
- **Capitolul 4 - Concluzii:** Prezintă un rezumat al rezultatelor obținute și validează îndeplinirea obiectivelor proiectului. Acest capitol include o evaluare experimentală a performanțelor hardware-software, o analiză tehnico-economică a soluției propuse și propune direcțiile viitoare de dezvoltare a sistemului.



## 2 Fundamente teoretice

Implementarea unui sistem de monitorizare pentru un microclimat extern, precum cel dintr-o incintă de fructificare a ciupercilor, necesită niște fundamente teoretice interdisciplinare. Acest capitol analizează principiile din spatele procesării Edge Computing, comportamentul dispozitivelor electronice în condiții de umiditate crescută, protocoalele de comunicație folosite în achiziția de date, sincronizarea bazelor de date, securitatea aplicațiilor mobile și tehnologiile utilizate.

### 2.1 Edge Computing în monitorizarea distribuită

În arhitecturile IoT clasice, dispozitivele periferice au un rol pasiv, limitându-se la colectarea datelor brute și transmiterea lor către un server centralizat unde se execută logica de control. Deși această abordare oferă avantaje mari în ceea ce privește capacitatea de stocare, ea introduce vulnerabilități majore în aplicațiile cu toleranță scăzută la erori.

Tranziția către Edge Computing presupune mutarea funcțiilor de calcul și a algoritmilor de decizie direct la sursă, exact acolo unde senzorii generează informația. Lucrarea [6] afirmă că procesarea centralizată în Cloud se lovește deseori de limitări de bandă, latențe imprevizibile în transmisie și riscul blocării complete a automatizării în cazul unei întreruperi de conectivitate. Prin introducerea unui nod computațional local, capabil să ruleze scripturi de analiză în timp real, sistemul capătă autonomie operațională. Buclele de control decurg local, eliminând timpii de propagare din rețea și asigurând continuitatea procesului biologic monitorizat, indiferent de starea conexiunii la internet.

### 2.2 Fiabilitatea componentelor electronice în medii cu umiditate ridicată

Cultivarea ciupercilor impune menținerea unei umidități relative constante în intervalul de 80-90% [8]. Acest mediu este considerat extrem de periculos pentru echipamentele informatice. Biologia fungică interzice scăderea acestui parametru, pentru unele specii, deoarece corpurile de fructificare nu dispune de bariere celulare protectoare împotriva deshidratării.

Din perspectiva ingineriei hardware, expunerea directă a circuitelor integrate la un astfel de microclimat poate duce la fenomene fizico-chimice ce pot distruge sistemul. Structural, umezeala acționează ca un plastifiant care scade temperatura de tranziție sticloasă  $T_g$  a unui circuit imprimat (PCB), amplificând vulnerabilitatea la stres termo-mecanic. Din punct de vedere electric, absorbția apei mărește constanta dielectrică  $D_k$  și factorul de disipare  $D_f$ , încetinind comutarea circuitelor și propagarea semnalelor. De asemenea, infiltrarea apei produce degradări ce favorizează formarea filamentelor de tip CFF. Adunate, aceste mecanisme, împreună cu coroziunea ionică, oxidarea cupru-

lui și delaminarea straturilor interne, conduc inevitabil la scurtcircuite și compromis definitiv unitatea de control [9].

Pentru a evita aceste riscuri, fără a apela la metode costisitoare de izolare industrială, teoria proiectării sistemelor de tip ES recomandă separarea fizică strictă a nodului de calcul de mediul controlat. Amplasarea logică a unității de procesare pe exteriorul incintei și pătrunderea în zona umedă doar prin intermediul magistrelor de date izolate protejează integritatea siliciului, asigurând stabilitatea pe termen lung a întregului sistem cu costuri minime de implementare.

## **2.3 Protocoale de comunicație folosite și achiziția digitală de date**

Sistemul de senzori necesită interfețe fizice capabile să asigure integritatea datelor pe liniile de transmisie expuse zgomotului electromagnetic de fundal. Astfel, acest proces este asigurat de către trei tehnologii importante, ce vor fi prezentate în rândurile care urmează.

### **2.3.1 Conversia Analog-Digital (ADC)**

Pentru a permite unui sistem electronic încorporat să interacționeze cu mărimi fizice din mediul real, care sunt continue în timp și amplitudine, este obligatorie utilizarea dispozitivelor de tip ADC. Achiziția digitală presupune eșantionarea și cuantificarea semnalului analogic brut, înainte de a fi transformat într-o valoare numerică [10]. Performanța și precizia acestui proces depind în mod direct de rezoluția convertorului. Dacă avem un spațiu de cuantificare mai extins, precum unul pe 16 biți, atunci convertorul permite detectarea unor variații de tensiune foarte fine [10]. Acest lucru este extrem de important în calibrarea senzorilor de gaz sau a celor de umiditate a solului, întrucât citirile lor pot fi afectate de zgomotul de fundal sau fluctuații parazite.

În funcție de arhitectura internă a convertorului, procesul de digitizare poate dura un interval de timp fix sau un interval variabil, ce depinde de valoarea eșantionată. Pentru dispozitivele ADC cu timp variabil, circuitele oferă un semnal de confirmare, denumit *done signal*, notificând procesorul în momentul în care datele sunt stabile și gata de preluare. Astfel, o interfață tipică de achiziție dispune, pe lângă intrările analogice, de alte două intrări digitale: un port de date, care permite citirea și scrierea regiștrilor interni și o intrare de ceas, responsabilă de sincronizarea și declanșarea următorului ciclu de conversie [10].

### 2.3.2 Protocolul I<sup>2</sup>C

Protocolul I<sup>2</sup>C sau Inter-Integrated Circuit este o magistrală serială sincronă, bidirecțională, bazată pe o arhitectură de tip master-slave, care utilizează doar două linii de semnal: *SDA* și *SCL*. La nivel electric, protocolul folosește configurații de tip open-drain sau open-collector, împreună cu rezistențe de pull-up conectate la tensiunea de alimentare, pentru a crea poarta logică *Wired-AND* [11].

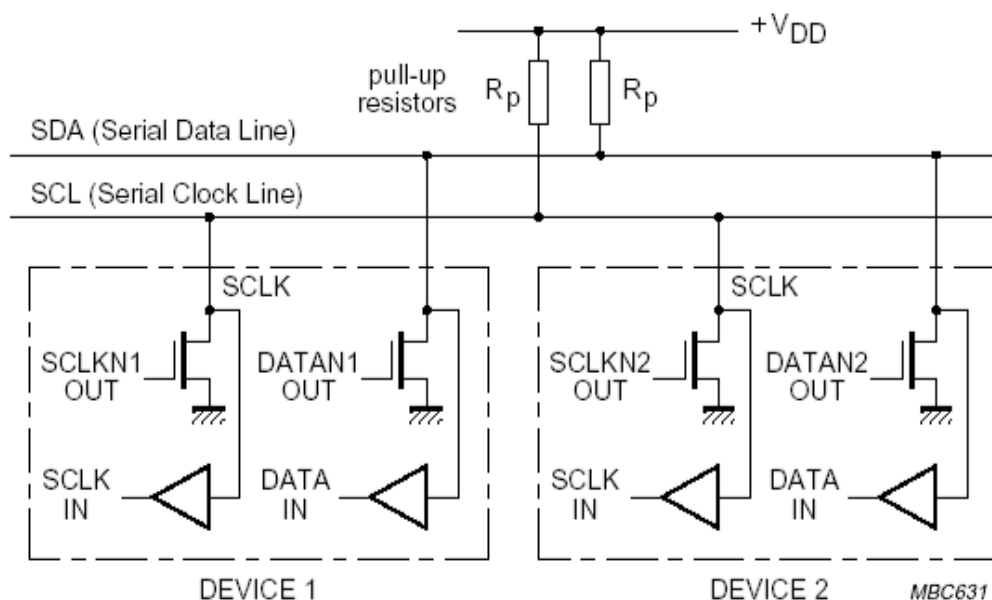


Figura 1: Exemplu de conectare a mai multor dispozitive I<sup>2</sup>C pe aceeași magistrală [12]

Din perspectiva arhitecturii sistemului, I<sup>2</sup>C folosește un mecanism de adresare pe 7 biți, permițând interconectarea și interogarea independentă a multiplexelor periferice, folosind aceleași două fire. Acest aspect reduce extrem de mult complexitatea structurală a conexiunilor la nivelul microcalculatorului. Totuși, specificațiile oficiale ale protocolului [11] limitează extinderea infinită a magistralei prin existența capacității parazite a liniilor (valoare maximă de 400 pF). Încadrarea în limita acestei variabile este esențială pentru a asigura viteze de transfer optime, precum 100 kbit/s în *Standard-mode* sau 400 kbit/s în *Fast-mode* [11].

### 2.3.3 Magistrala 1-Wire

1-Wire este un standard de comunicație digitală, ce utilizează o arhitectură de tip master-slave și operează în regim *half-duplex* bidirecțional printr-o singură linie de date și o referință de masă [13].

Un aspect fundamental îl reprezintă capacitatea senzorilor de a funcționa fără un pin de alimentare dedicat, aceștia extrăgând energia necesară din linia de date prin tehnica alimentării parazite [13]. Pentru a permite conectarea mai multor dispozitive pe aceeași magistrală, fiecare senzor are un cod de identificare unic pe 64 de biți, care acționează ca o adresă de rețea [13]. Rezistența proto-

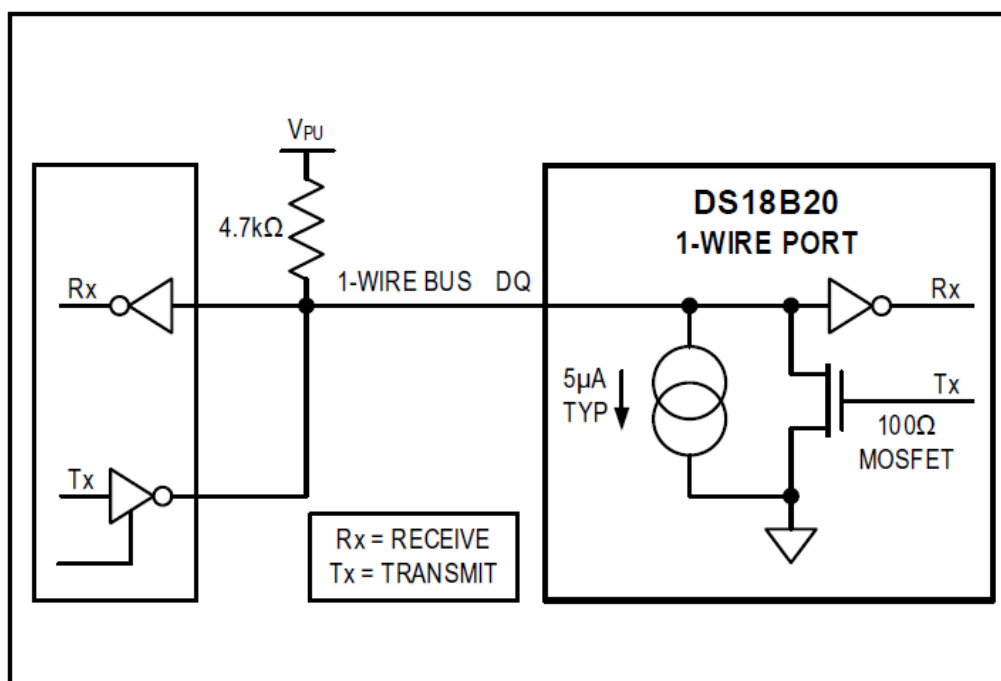


Figura 2: Configurația hardware a unui dispozitiv 1-Wire [14]

colului în medii corozive este asigurată de mecanismul de reconectare. Astfel, la orice pierdere temporară de contact sau tensiune, senzorii intră în stare de *reset*, emițând automat un puls de prezență la restabilirea tensiunii [13]. În acest fel, se asigură o citire precisă a datelor pe distanțe relativ mari, folosind un singur pin GPIO al procesorului.

## 2.4 Modele de stocare și mesagerie în timp real

Funcționarea optimă a unui sistem IoT distribuit, bazat pe o arhitectură Edge-Cloud, depinde de eficiența cu care datele sunt arhivate și transmise către aplicațiile client. În această secțiune sunt analizate fundamentele teoretice și deciziile arhitecturale care stau la baza infrastructurii de date a proiectului, argumentând necesitatea tranziției către modele non-relaționale de stocare și adoptarea unor metode asincrone pentru a garanta o comunicare bidirecțională cu latență minimă.

### 2.4.1 Avantajele bazelor de date NoSQL

Bazele de date NoSQL sunt utilizate pentru a gestiona cantități mari de date structurate, semi-structurate sau nestructurate cu ajutorul unor scheme flexibile [15]. Chiar dacă modelul bazelor de date relaționale, oferit de SQL, este unul dintre cele mai cunoscute, industria a simțit totuși necesitatea creării unor alternative la SQL, întrucât exista nevoia de scalabilitate orizontală mult mai mare și dorința de a stoca structuri de date care nu se potrivesc ușor în tabele bidimensionale [16]. O provocare majoră a sistemelor tradiționale este ceea ce lucrarea [16] o numește ca "impedance mis-

match”, mai exact, discrepanța dintre obiectele create în codul aplicației și modelul relațional al bazei de date. Prin utilizarea unui model orientat pe documente, bazat pe formatul JSON, structurile de date se aliniază mult mai natural cu logica de programare.

Un avantaj esențial al modelului de documente este abordarea de tip *schema-on-read*, unde structura datelor e interpretată doar la momentul citirii [16]. Această flexibilitate este necesară pentru sistemele IoT aflate în evoluție, deoarece formatul pachetelor de date poate suferi modificări odată cu calibrarea sau adăugarea de noi senzori în ferma de ciuperci, fără a migra întreaga bază de date. Mai mult, modelul NoSQL oferă o localizare superioară a datelor, întrucât informațiile ambientale relevante pentru un anumit moment de timp sunt stocate într-un singur document JSON. Astfel, citirea parametrilor se face printr-o singură interogare extrem de rapidă, evitând operațiile de tip “JOIN”, care afectează performanța în sistemele SQL [16].

#### 2.4.2 Tehnici de sincronizare în timp real a datelor

Monitorizarea la distanță a stării sistemului impune o sincronizare cu latență minimă. Analizând protocoalele de comunicație web, lucrarea [17] subliniază faptul că modelul tradițional HTTP, bazat pe modelul request-response sau tehnicile de interogare repetată, introduce un consum ineficient de lățime de bandă și latențe inacceptabile pentru un sistem de monitorizare în timp real. Clientul este forțat să deschidă constant conexiuni noi doar pentru a “întreba” serverul dacă au apărut date noi [17]. Pentru a rezolva această problemă, arhitecturile moderne de sincronizare (precum Firebase Realtime Database) se bazează pe standardul WebSockets [18]. Acesta stabilește o conexiune TCP persistentă, full-duplex și bidirecțională între client și server [17].

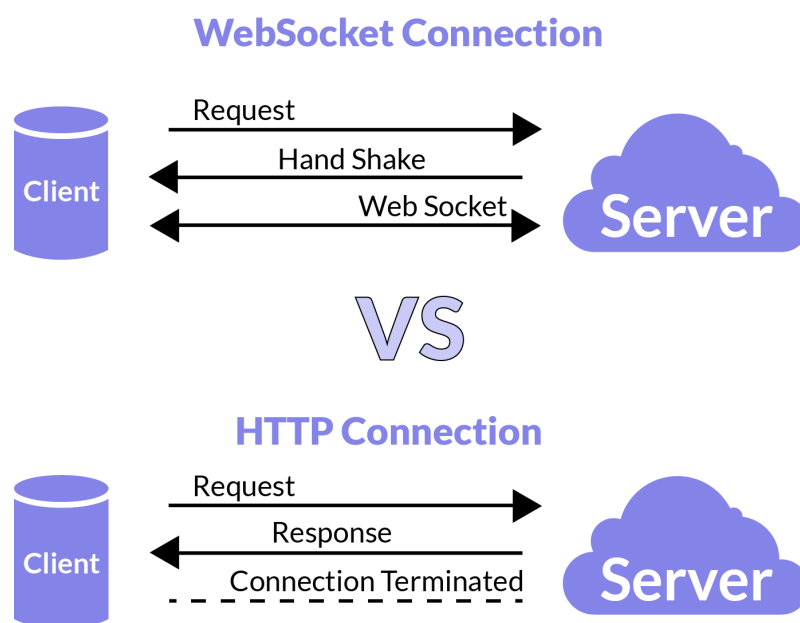


Figura 3: Diferența dintre metoda Polling și WebSocket [19]

Peste această infrastructură fizică de comunicație este implementat modelul de mesagerie *Publish-Subscribe*. În acest model asincron, componentele sistemului funcționează ca entități complet separate. Astfel, nodul de achiziție local acționează ca un publicator (*publisher*) care trimite modificările de telemetrie către baza de date, fără a cunoaște identitatea sau numărul clienților finali. La rândul lor, aplicațiile software deschid un canal permanent de ascultare și se abonează (*subscribe*) la ramuri specifice din arborele JSON. În momentul în care parametrii de mediu se modifică, serverul trimite automat și instantaneu noile valori către toate dispozitivele mobile abonate, garantând o sincronizare fluentă și un trafic de rețea optimizat [17].

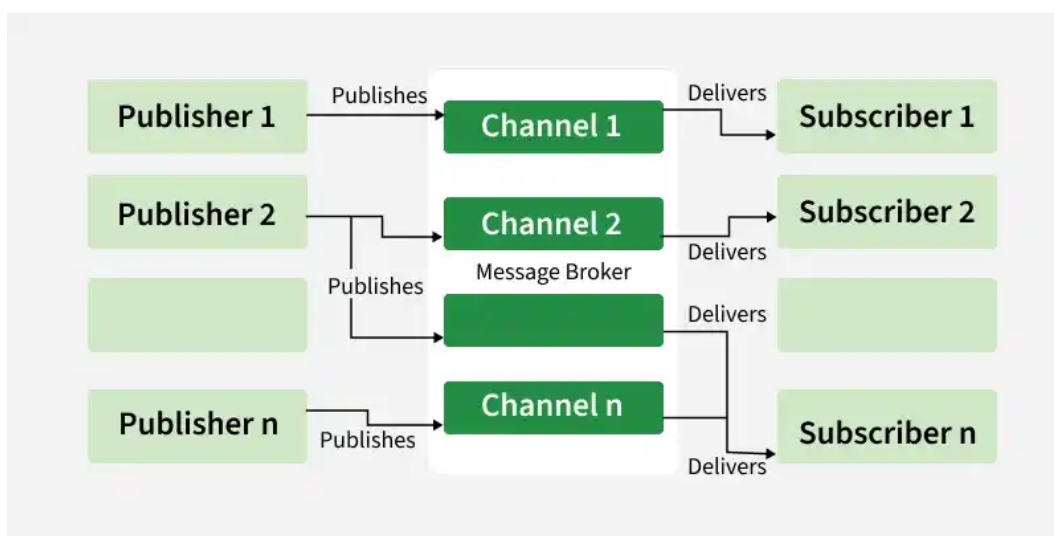


Figura 4: Modelul de mesagerie Publish-Subscribe [20]

Acest canal bidirecțional și structura flexibilă JSON permit, de asemenea, o monitorizare asincronă a echipamentului fizic. Printr-un mecanism de monitorizare a stării conexiunii sistemului, nodul de achiziție scrie secvențial un marcaj de timp la o anumită ramură a bazei de date. Absența actualizării acestui parametru într-un anumit interval de timp indică pierderea conexiunii, permițând sistemului să alerteze utilizatorul că nodul central s-a închis sau conexiunea la internet s-a pierdut [21].

## 2.5 Securitatea aplicațiilor mobile și modelul Zero-Trust

Interfața de utilizator, dezvoltată prin *framework-uri* native hibride, simplifică distribuția aplicației pe diverse sisteme de operare (Android/iOS) prin utilizarea unei baze de cod unificate ce afișează componente direct la nivelul hardware-ului telefonului [22].

Întrucât aplicația permite modificarea de la distanță a pragurilor critice de mediu ale culturii, securitatea accesului devine o prioritate absolut necesară. Arhitecturile software moderne au abandonat modelul tradițional de securitate bazat pe perimetru (care presupune că orice cerere venită din interiorul unei rețele sau din partea unei aplicații instalate este legitimă) în favoarea conceptului de

securitate Zero-Trust [23]. Principiul fundamental al acestui model este simplu: o entitate trebuie să presupună că nu există încredere implicită și să analizeze și să evalueze continuu riscurile, aplicând întotdeauna protecții pentru a le atenua [23]. Tradus în logica aplicației mobile, acest model impune ca nicio acțiune critică (precum transmiterea de valori noi pentru pragurile critice) să nu fie procesată exclusiv pe baza existenței unui token de sesiune stocat în memorie. Aplicația solicită re-autentificarea explicită a utilizatorului prin mecanisme hardware biometrice locale, precum utilizarea amprentei sau scanarea facială, înainte de a intra în aplicație, blocând astfel tentativele de acces neautorizat în cazul pierderii sau compromiterii dispozitivului mobil [24].

## 2.6 Tehnologii, platforme și medii de dezvoltare utilizate

### 2.6.1 Single-Board Computer (SBC)

Un Single-Board Computer, din punct de vedere tehnic, este un calculator de dimensiuni mici, ale căror componente principale, precum procesorul și memoria RAM sunt integrate într-un singur SoC. Acest dispozitiv poate fi folosit în aplicații de calcul, robotică sau IoT [25].

Din punct de vedere al arhitecturii, un Single-Board Computer oferă o arhitectură hardware hibridă, întrucât dispune atât de puterea de procesare necesară rulării unui sistem de operare complet, însă menține și o interfață fizică la nivel scăzut prin intermediul porturilor GPIO [26]. Această arhitectură facilitează procesul de Edge Computing, unde filtrarea și procesarea datelor se realizează local, înainte de transmisia acestora, reducând latența și lățimea de bandă necesară comunicării cu platformele cloud [27]. În acest fel, sistemul încorporat devine un nod inteligent de decizie, depășind limitările arhitecturilor centralizate [28].

### 2.6.2 Limbajul de programare Python

Python este un limbaj de programare de nivel înalt, interpretat și open-source, folosit în principal pentru dezvoltare de aplicații web și software, data science și machine learning (ML) [29]. Acesta include un set de caracteristici de programare orientată pe obiecte (OOP), procedurală și funcțională care sunt folositoare pentru menținerea lizibilității codului. Datorită conceptului de "dynamic typing" (o singură variabilă poate să conțină date de diferite tipuri pe parcursul rulării codului) și a gestionării automate a memoriei, limbajul facilitează prototiparea rapidă a programelor [30].

Pe lângă flexibilitatea sintactică, un avantaj major al ecosistemului Python îl reprezintă biblioteca standard a acestui limbaj de programare, care conține module încorporate (dezvoltate în C sau Python) ce oferă acces la o multitudine de funcționalități de sistem sau soluții standardizate [31]. Pe lângă biblioteca standard, funcțiile limbajului Python pot să fie extinse cu ajutorul unei colecții mari de bib-

lioteci "third-party", ce simplifică sarcinile de asociere și procesare a datelor, incluzând serializarea și deserializarea formatelor structurate, precum JSON [30]. De asemenea, limbajul oferă suport avansat pentru programarea asincronă, funcționalitate critică în arhitecturile IoT. Utilizarea operațiilor de intrare/ieșire non-blocante (*non-blocking I/O*) permite gestionarea concurentă a multiplelor sarcini, optimizând astfel consumul de resurse pe platformele cu putere de calcul limitată [32].

Deși este un limbaj de nivel înalt, arhitectura limbajului Python permite interfațarea cu module, biblioteci și registre hardware scrise în limbaje de nivel jos, precum C sau C++. Această caracteristică oferă o modalitate de a separa logica de aplicație de controlul direct al pinilor GPIO, facilitând comunicarea cu senzorii fizici prin intermediul protocoalelor seriale. Astfel, performanța sau viteza de execuție la nivel hardware nu sunt afectate [30].

### 2.6.3 Limbajul de programare TypeScript

Creat de către compania Microsoft, TypeScript este un limbaj de programare open-source, ce pune accentul pe verificarea tipurilor de date în faza de compilare, spre deosebire de JavaScript. TypeScript este adesea descris ca "o extensie a limbajului JavaScript". Acest lucru înseamnă că orice cod scris în JavaScript care rulează, va funcționa și în TypeScript, facilitând o migrare ușoară către o bază de cod care valorifică funcționalități avansate de programare orientată pe obiecte [33].

O caracteristică avansată a limbajului o reprezintă decoratorii. La bază, aceștia sunt un tip special de declarații, care pot fi atașate claselor, metodelor sau proprietăților. Prefixate cu simbolul "@", aceste funcții sunt folosite pentru a modifica comportamentul elementului decorat [34].

Un alt concept fundamental introdus de TypeScript îl reprezintă utilizarea interfețelor și a aliașilor de tip (*type aliases*). Acestea sunt tipuri de obiecte, cu ajutorul cărora putem să grupăm atât proprietățile, cât și tipurile de date ale acestora. În contextul sistemelor distribuite care comunică asincron cu baze de date NoSQL sau API-uri externe, aplicațiile preiau constant pachete sub formă de obiecte JSON. Astfel, interfețele se asigură că acest obiecte primite din rețea respectă cu exactitate structura predefinită în cod. În TypeScript, orice discrepanță de formatare este semnalată de compilator, eliminând astfel erorile critice la timpul de execuție [34].



### 2.6.4 React Native și platforma Expo

React Native este un framework bazat pe limbajul JavaScript, ce are ca scop dezvoltarea de aplicații mobile. Acesta permite programatorilor să construiască software destinat platformelor iOS și Android, utilizând o bază de cod unificată [22]. Spre deosebire de framework-urile hibride vechi, precum Apache Cordova, care rulează o aplicație web în interiorul unui container ce interacționează cu sistemul de operare (OS) al telefonului [35], React Native apelează direct API-urile native ale sistemului de operare (Objective-C pentru iOS sau Java/Kotlin pentru Android) [22]. Un avantaj important al React Native îl reprezintă sintaxa și funcționalitățile similare cu framework-ul pentru dezvoltarea aplicațiilor web, React. Astfel, un programator care are experiență în lucrul cu React, poate să migreze foarte ușor la React Native, dacă vrea să dezvolte aplicații mobile cross-platform [22].

Pentru a eficientiza la maxim ciclul de dezvoltare, acest framework este adesea integrat cu ecosistemul open-source Expo. Acesta funcționează ca un strat de abstractizare peste complexitatea dezvoltării mobile native, eliminând necesitatea configurării manuale a mediilor de dezvoltare Android Studio sau Xcode. Platforma gestionează implicit modulele native și oferă un set generos de API-uri deja configurate, permițând actualizări dinamice și o experiență de dezvoltare stabilă [36].

### 2.6.5 Firebase

Gestionarea tradițională a unei baze de date sau a unui server poate să implice eforturi destul de mari când vine vorba de mentenanță și securitate. Pentru a le minimiza, dezvoltatorii optează pentru folosirea unui model BaaS (backend ca serviciu) [37]. O platformă ce adoptă acest model este Firebase, dezvoltată de către compania Google, ce are rolul de a ușura munca depusă de programator pentru mentenanța infrastructurii fizice. Pe lângă asta, Firebase oferă o multitudine de instrumente integrate, găzduite într-un mediu Cloud [38].

O funcționalitate critică a acestui model este baza de date în timp real de tip NoSQL, unde informațiile sunt stocate sub formă de arbori JSON. O caracteristică importantă a acestei arhitecturi este mecanismul de sincronizare asincronă. Spre deosebire de sistemele clasice, unde clientul trebuie să interogheze constant serverul, baza de date menține o conexiune persistentă prin WebSockets [18]. Astfel, sistemul trimite automat și instantaneu noile date către toți clienții conectați, ori de câte ori o valoare se modifică.

Securitatea tranzacțiilor de date reprezintă o componentă fundamentală integrată în ecosistem. În arhitecturile clasice, securitatea necesită dezvoltarea unui strat software intermediar (*middleware*) pentru a valida fiecare cerere. Firebase elimină această necesitate prin implementarea unor reguli de securitate declarate la nivelul bazei de date [39]. De asemenea, platforma dezvoltată de către Google

oferă și integrarea nativă cu servicii de autentificare a utilizatorilor, prin Firebase Authentication. Astfel, platforma evaluează identitatea utilizatorului și permisiunea acestuia în timp real la baza de date, blocând orice tentativă de acces neautorizat [38].

### 2.6.6 Ngrok

În dezvoltarea și implementarea sistemelor IoT distribuite, o provocare principală o reprezintă accesibilitatea nodurilor periferice din rețelele exterioare. De cele mai multe ori, microcalculatoarele locale își desfășoară activitatea în spatele unor routere la care sunt conectate direct. Această structură împiedică accesarea directă a dispozitivului din afara rețelei locale, deoarece unitatea de calcul nu posedă o adresă IP publică rutabilă.

Pentru a trece peste această limitare fără a recurge la configurări nesecurizate de redirectionare a porturilor (*Port Forwarding*) la nivelul routerului, în cadrul proiectului s-a integrat platforma Ngrok. Utilizarea platformei Ngrok în arhitecturi IoT bazate pe microcalculatoare din familia Raspberry Pi reprezintă o soluție validată în lucrarea [40] pentru facilitarea monitorizării de la distanță și ocolirea restricțiilor de rețea locale.

Ngrok este o platformă care permite partajarea securizată a serverelor locale prin internet cu ajutorul unor tunele inversate [41]. Din punct de vedere funcțional, un agent Ngrok rulează local pe sistemul de operare al dispozitivului și inițiază o conexiune continuă de ieșire către infrastructura Cloud, oferită de Ngrok [42].

### 2.6.7 GitHub

GitHub este o platformă web colaborativă construită în jurul sistemului Git, destinată controlului versiunilor, colaborării și gestionării proiectelor software. În timp ce Git gestionează la nivel de nucleu controlul versiunilor prin urmărirea și administrarea strictă a modificărilor de cod, GitHub funcționează ca un mediu de găzduire a repository-urilor, oferind instrumente avansate pentru munca în echipă și contribuțiile de la distanță.

Elementul central al arhitecturii îl reprezintă *repository-ul*, ce reprezintă un spațiu dedicat în care sunt stocate fișierele și directoarele unui proiect. Acesta menține istoricul complet al modificărilor folosind Git, asigurând astfel monitorizarea fiecărei linii de cod scrise de-a lungul dezvoltării unui proiect.

Pentru a asigura stabilitatea codului aflat în producție, platforma utilizează conceptul de ramuri (*branches*). Acestea permit dezvoltatorilor să lucreze în paralel la funcționalități noi sau la remedierea erorilor, complet separați de codul principal. Odată ce o modificare este finalizată pe o ramură

secundară, se inițiază o cerere de integrare (*Pull Request*). Acest mecanism este esențial pentru menținerea calității software-ului, deoarece impune revizuirea codului (*code review*), facilitând discuțiile și feedback-ul înainte ca modificările propuse să fie implementate definitiv [43].

În cadrul acestei lucrări, platforma GitHub a reprezentat infrastructura principală pentru stocarea, versionarea și managementul codului sursă. S-a folosit o arhitectură de stocare de tip *monorepo*, care a permis centralizarea ambelor componente ale sistemului în același spațiu: logica de control Edge dezvoltată în Python și aplicația mobilă cross-platform creată în React Native. Din punct de vedere al versionării, s-a folosit un model liniar, în care integrările de funcționalități s-au realizat direct pe ramura principală. Această abordare a asigurat o sincronizare constantă între evoluția codului de pe sistemul de achiziție și actualizările interfeței mobile. Pentru a consolida acest model și a defini punctele de referință sigure, s-a integrat sistemul de *Releases* nativ platformei. Prin intermediul acestuia, au fost marcate versiunile stabile pregătite pentru mediul de producție.

## 3 Proiectarea și implementarea sistemului

Acest capitol prezintă în detaliu contribuțiile aduse în rezolvarea temei de proiectare. Este detaliată trecerea de la conceptele teoretice la realizarea practică a prototipului, argumentând alegerea fiecărei componente hardware și a soluțiilor software utilizate pentru a construi un sistem IoT stabil, capabil să controleze și să monitorizeze microclimatul fungic.

### 3.1 Arhitectura generală a sistemului

Pentru a îndeplini obiectivele propuse, a fost proiectată o arhitectură de sistem modulară, prezentată în **Figura 5**, structurată pe trei niveluri distincte: **nivelul de achiziție** (format din senzori), **nivelul de decizie și procesare locală** și **nivelul de stocare și interacțiune cu utilizatorul**. Această structură asigură o separare a responsabilităților și o scalabilitate ridicată a întregului ansamblu.

Pentru a oferi o imagine de ansamblu asupra schimbului de date dintre nivele, responsabilitățile au fost împărțite astfel:

- **Nivelul de achiziție:** Acest strat reprezintă interfața directă cu ferma de ciuperci. Funcția sa principală este de a prelua mărimi fizice (temperatură, umiditate relativă, umiditatea solului, nivelul de iluminare și concentrația nivelului de CO<sub>2</sub>) și de a le converti în semnale electrice digitale sau analogice.
- **Nivelul de decizie și procesare locală:** Acest nivel este deservit de către microcalculatorul Raspberry Pi 5, acționând ca nodul inteligent al sistemului fizic. Aici sunt rulate scripturile Python care realizează interogarea secvențială a senzorilor (prin protocoalele I<sup>2</sup>C și 1-Wire), filtrarea zgomotului de fundal din semnalele analogice și afișarea locală pe ecranul OLED. În plus, la acest nivel a fost integrat și o cameră web USB și un server HTTP local dedicat. Pentru a preveni uzura cardului de memorie, pozele sunt capturate la fiecare 5 secunde într-un fir de execuție și stocate direct în memoria RAM (în directorul /tmp), asigurând o latență minimă la citire.
- **Nivelul de stocare și interacționare cu utilizatorul:** Reprezintă componenta distribuită a arhitecturii. Datele obținute local sunt transmise printr-o conexiune la Internet, utilizând interfața rețelei wireless locale a microcalculatorului, către platforma Firebase Realtime Database. Acest nivel gestionează istoricul de date și facilitează comunicarea bidirecțională cu aplicația mobilă cross-platform, dezvoltată în React Native. În paralel, fluxul multimedia de la cameră este expus securizat pe internet prin intermediul serviciului de tunelare Ngrok, care generează

un endpoint public protejat prin HTTPS. Prin intermediul acestei interfețe externe, utilizatorul poate monitoriza starea sistemului în timp real, poate primi alerte contextuale, poate ajusta de la distanță pragurile critice de funcționare ale sistemului și poate urmări evoluția biologică a ciupercilor prin ecranul de vizualizare în timp real.

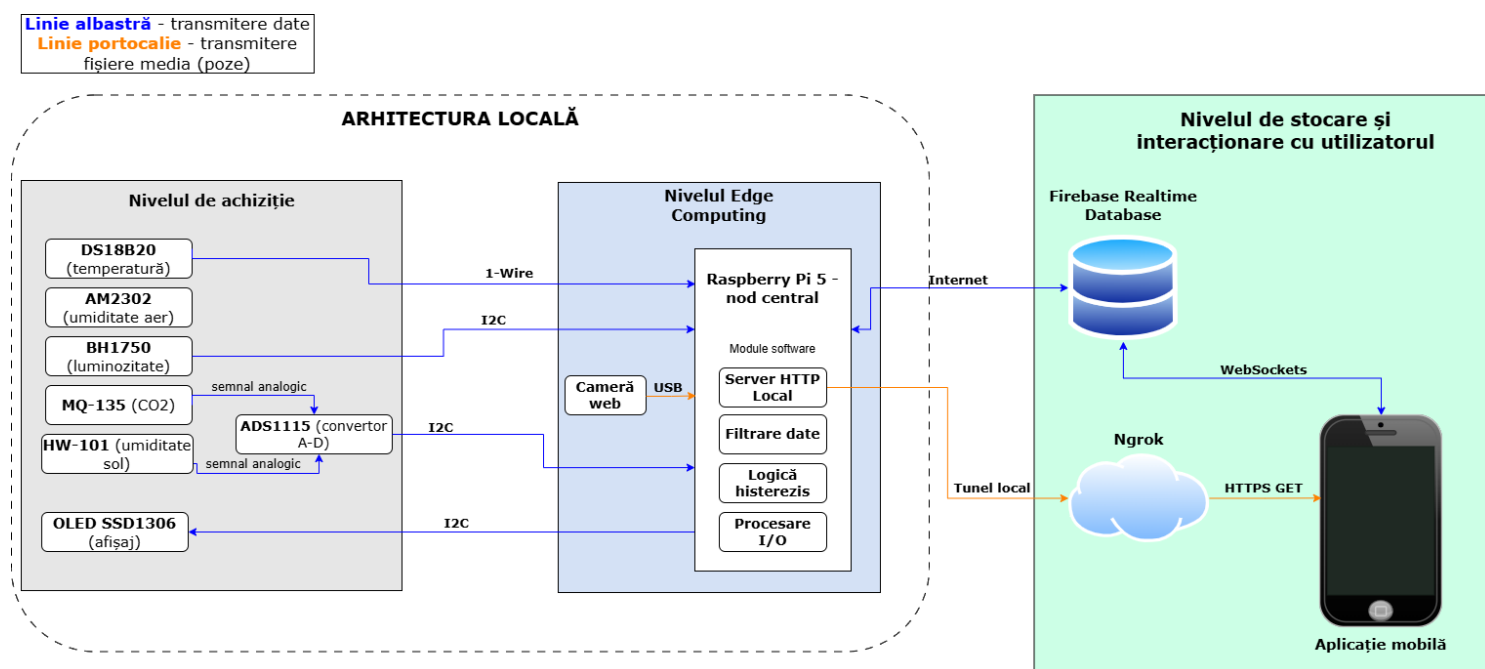


Figura 5: Arhitectura generală a sistemului

## 3.2 Proiectarea hardware și alegerea componentelor

Proiectarea nivelului hardware a necesitat o selecție atentă a componentelor, având ca și criteriu principal rezistența într-un mediu dominat de o umiditate relativă constantă de 80-90% și necesitatea comunicării prin protocoale seriale cu timpi mici de răspuns.

### 3.2.1 Alegerea unității centrale de procesare

Având în vedere necesitatea implementării unei platforme capabile de Edge Computing, menite să elimine dependența exclusivă de un server extern, s-a optat pentru utilizarea microcalculatorului Raspberry Pi 5.

Din punct de vedere al caracteristicilor tehnice, placa este echipată cu sistemul pe cip (SoC) Broadcom BCM2712, integrând un procesor quad-core pe arhitectură ARM Cortex-A76, cu o frecvență de 2.4 GHz, alături de o memorie SDRAM LPDDR4X de **8 GB**. În contextul unui sistem IoT distribuit, o caracteristică tehnică importantă a acestei plăci o reprezintă modulul de conectivitate wireless integrat (Dual-band 802.11ac Wi-Fi), care elimină necesitatea utilizării unor adaptoare de rețea externe

pentru comunicarea cu platforma Cloud [44].

Decizia folosirii microcalculatorului Raspberry Pi 5 a fost luată considerând mai mulți factori. În comparație cu microcontrolerele clasice, precum cele din gama Arduino, microcalculatoarele din familia Raspberry Pi permit rularea unui sistem de operare Linux complet (Raspberry Pi OS). Acest mediu este esențial pentru execuția concurentă a scripturilor Python 3 destinate citirii senzorilor prin pinii GPIO, interogării bazelor de date prin conexiune criptată și gestionării logicii de control termic. Mai mult, prin prezența magistralelor I<sup>2</sup>C și 1-Wire la nivelul portului GPIO de 40 de pini a simplificat semnificativ arhitectura fizică a plăcii perforate de interfațare.

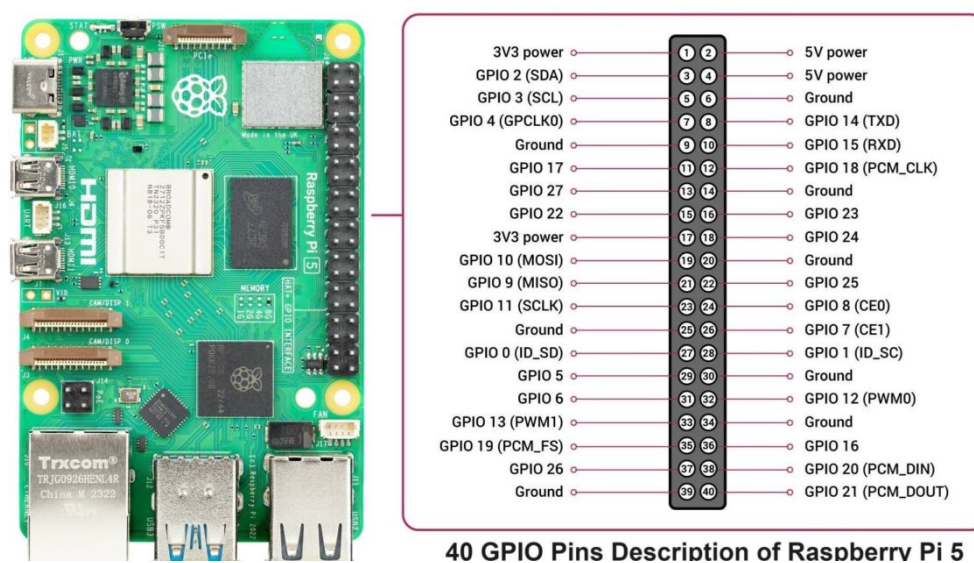


Figura 6: Pinout Raspberry Pi 5 [45]

### 3.2.2 Integrarea convertorului analog-digital

O provocare arhitecturală a unității Raspberry Pi o reprezintă absența unui convertor analog-digital integrat la nivelul pinilor GPIO, acesta fiind capabil să citească doar semnale digitale. Deoarece monitorizarea concentrației de CO<sub>2</sub> și a umidității solului impune utilizarea unor senzori cu ieșire analogică (MQ-135 și HW-101), s-a folosit un circuit integrat intermediar, care să realizeze această sarcină.

Soluția aleasă a fost modulul ADS1115, un convertor analog-digital pe 16 biți, recunoscut pentru precizia sa ridicată. Din punct de vedere al configurației hardware, cipul pune la dispoziție un multiplexor de intrare capabil să gestioneze 4 canale cu referință comună *single-ended* (A0-A3) sau 2 canale diferențiale [46]. Această configurație a permis conectarea simultană și independentă a ambilor senzori analogici pe primele două canale, lăsând alte două porturi disponibile pentru o viitoare extindere a sistemului.

Un factor important în selecția acestui modul a fost intervalul mare a tensiunii de operare (2.0V - 5.5V) [46]. Prin alimentarea convertorului direct la borna de 3.3V a plăcii Raspberry Pi, se asigură compatibilitatea nivelurilor logice cu pinii GPIO. Această abordare elimină necesitatea utilizării unui circuit adițional de tip adaptor de nivel logic (*logic level shifter*), reducând complexitatea circuitului și protejând procesorul împotriva supratensiunilor.

Conectarea cu unitatea centrală se realizează prin magistrala serială I<sup>2</sup>C, ocupând doar doi pini: SDA și SCL, lăsând restul pinilor liberi pentru alte funcționalități. La nivel de configurație, adresa fizică a cipului a fost fixată la 0x48 prin conectarea la masă (GND) a pinului de adresă (ADDR), prevenind astfel orice conflict de comunicare pe magistrală [46].

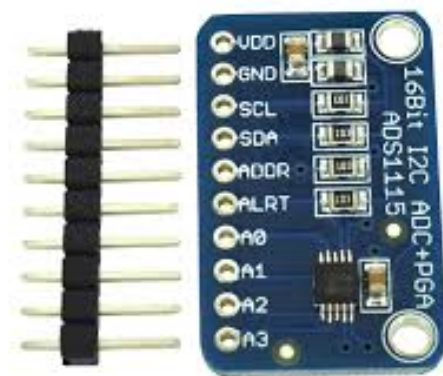


Figura 7: Convertorul Analog-Digital ADS1115 [47]

### 3.2.3 Motivarea alegerii senzorilor utilizați

Eficiența buclei de control și stabilitatea microclimatului fungic depind în mod direct de calitatea și precizia datelor de intrare. Prin urmare, selecția ansamblului de senzori a urmărit obținerea unei precizii ridicate într-un mediu predispus la condens, precum și simplificarea configurației de cablare prin utilizarea protocoalelor seriale de comunicație.

1. **Senzorul de temperatură (DS18B20)** - Pentru monitorizarea cu precizie a temperaturii aerului, s-a folosit senzorul digital DS18B20. Din punct de vedere arhitectural, cipul intern dispune de un convertor analog-digital propriu, capabil să ofere o rezoluție programabilă între 9 și 12 biți. Interfațarea cu Raspberry Pi 5 se realizează prin protocolul de comunicație *1-Wire*, care permite transmiterea datelor și a semnalului de ceas pe un singur fir (pinul DQ). O cerință hardware obligatorie a acestei magistrale, implementată pe placa de dezvoltare, a fost adăugarea unei rezistențe de pull-up de 4.7 kΩ între linia de alimentare și linia de date. Aceasta asigură

menținerea unui nivel logic ridicat pe magistrală în starea de repaus, permițând microcalculatorului să interogheze senzorul folosind adresa sa unică pe 64 de biți, stocată permanent în memoria ROM internă [14].

2. **Senzorul de temperatură și umiditate relativă (AM2302)** - Deoarece umiditatea aerului trebuie menținută strict în intervalul 80-90%, a fost necesar un senzor ce poate să opereze la saturații mari fără a-și pierde calibrarea. Modulul AM2302 integrează un element capacitiv sensibil la polimeri pentru umiditate și un termistor NTC pentru temperatură. Spre deosebire de senzorii analogici, acesta utilizează un semnal digital pe o singură magistrală de date. Din punct de vedere al specificațiilor hardware, pinul de date a fost conectat direct la un pin GPIO al unității centrale, senzorul transmițând un pachet de date de 40 de biți, ce include atât valorile măsurate, cât și un bit de paritate "checksum", folosit pentru validarea integrității pachetului [48].
3. **Senzorul de iluminare (BH1750)** - Lumina joacă câteva roluri importante în dezvoltarea ciupercilor, mai exact prin formarea structurilor reproductive și în răspunsurile fototropice ale acestor structuri [8]. Pentru a monitoriza acest factor, s-a utilizat modulul digital BH1750. Configurarea sa hardware se bazează pe o fotodiodă internă cuplată la un ADC pe 16 biți, care elimină necesitatea unor calcule complexe, livrând direct unitatea de măsură a iluminării (Lux). Conexiunea s-a realizat prin magistrala I<sup>2</sup>C. Pinul de adresare al modulului a fost conectat la masă (GND), stabilind adresa fixă 0x23 pentru comunicarea cu procesorul central [49].
4. **Senzorul analogic de gaze și calitatea aerului (MQ-135)** - Monitorizarea dioxidului de carbon (CO<sub>2</sub>) este absolut necesară pentru a preveni malformațiile corpurilor de fructificare [8]. Modulul MQ-135 se bazează pe un element chimic sensibil din dioxid de staniu (SnO<sub>2</sub>), a cărui rezistență electrică scade pe măsură ce concentrația gazelor țintă din aer crește. Arhitectura sa necesită un curent constant pentru un filament de încălzire (*heater*), alimentat la 5V. Deoarece ieșirea sa este strict proporțională cu concentrația de gaz și se prezintă sub forma unei variații de tensiune analogice pe pinul AO, acest pin a fost conectat către canalul de intrare AO al ADC-ului extern [50]. Conversia datelor în PPM se realizează ulterior software, pe baza calculării rezistenței în aer curat (R<sub>0</sub>).
5. **Senzorul analogic de umiditate a solului (HW-101)** - Pentru a monitoriza gradul de hidratare a substratului, a fost integrat senzorul de umiditate HW-101. Spre deosebire de modelele tradiționale de pe piață, precum FC-28, care evaluează umiditatea prin variația rezistenței electrice dintre doi electrozi expuși [51], acest modul utilizează principiul detecției capacitive. Această



modificare rezolvă problema coroziunii rapide a circuitului la contactul prelungit cu un sol umed, extinzând considerabil fiabilitatea și durata de viață a componentei în fermă. Din punct de vedere hardware, senzorul suportă o tensiune de lucru între 3.3V și 5.5V, integrând un stabilizator de tensiune care îi asigură compatibilitatea directă cu nivelurile logice ale unității de control. Modulul generează o tensiune de ieșire analogică cuprinsă în intervalul 0-3V. O caracteristică tehnică importantă pentru calibrarea software a acestui senzor este faptul că tensiunea citită este invers proporțională cu nivelul de umiditate din sol; astfel, dacă tensiunea la ieșire are un nivel ridicat, solul este uscat sau aproape uscat [52]. Deoarece Raspberry Pi necesită un modul extern de conversie pentru procesarea semnalelor analogice, pinul analogic al senzorului a fost conectat direct la canalul A1 al convertorului ADS1115, permițând digitizarea și eșantionarea precisă a datelor.

### 3.2.4 Integrarea interfeței vizuale locale

Pentru a asigura o interacțiune directă și rapidă a utilizatorului cu sistemul fizic, fără a depinde de aplicația mobilă sau de platforma Cloud, arhitectura a fost completată cu un modul de afișaj local. S-a optat pentru un ecran OLED cu diagonala de 0.96 inch, controlat de driverul integrat SSD1306.

Alegerea tehnologiei OLED în favoarea ecranelor clasice LCD a fost motivată de câteva avantaje tehnice, printre care contrastul foarte ridicat (fiecare pixel emite propria lumină, nefiind nevoie de folosirea unui backlight, utilizat în tehnologia LCD) și consumul energetic extrem de redus [53].

Din perspectiva conectării, modulul SSD1306 utilizează magistrala serială I<sup>2</sup>C. Alimentarea s-a realizat la tensiunea logică de 3.3V, iar pentru a asigura comunicarea corectă pe magistrală, scriptul software a fost configurat să apeleze adresa 0x3C, specifică acestui modul.

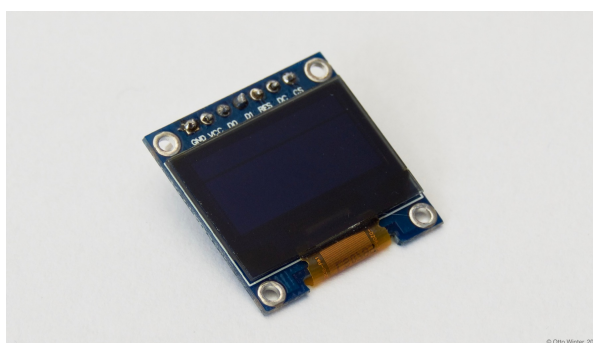


Figura 8: Ecranul OLED SSD1306 [54]

### 3.2.5 Integrarea dispozitivului de monitorizare vizuală

Pentru a permite monitorizarea vizuală directă a evoluției ciupercilor de către utilizator, arhitectura hardware a fost extinsă prin integrarea unei camere web generice. Dispozitivul utilizează o interfață nativă USB, fiind conectat direct la unul dintre porturile USB ale microcalculatorului Raspberry Pi 5.

Alegerea unei camere cu interfață USB a oferit un avantaj din punct de vedere al ingineriei sistemului, întrucât utilizarea protocolului standard UVC asigură recunoașterea imediată a dispozitivului de către kernel-ul sistemului de operare Linux. Acest lucru elimină complet necesitatea instalării sau dezvoltării unor drivere de nivel scăzut, reducând timpul de dezvoltare. Camera a fost configurată pentru a captura cadre la o rezoluție standard (1024x576), asigurând un echilibru între claritatea detaliilor vizuale ale culturii de ciuperci și dimensiunea redusă a fișierului de memorie rezultat. Acest lucru este necesar pentru menținerea unei latențe minime în timpul transmisiunii ulterioare prin rețea. Din punct de vedere energetic și structural, camera este alimentată direct prin magistrala USB a plăcii, fiind complet independentă de circuitul electronic principal dezvoltat pe placa perforată.

### 3.2.6 Realizarea circuitului fizic

Trecerea de la stadiul de alegere a componentelor la un prototip funcțional, capabil să reziste în condițiile de operare ale unei incinte de cultivare a ciupercilor, a necesitat proiectarea unui circuit ce cuprinde toate modulele și dispozitivele descrise anterior. Obiectivul principal al acestei etape a fost asigurarea stabilității conexiunilor electrice și gestionarea corectă a nivelurilor de tensiune.

Pentru a evita instabilitatea firelor de conexiune des întâlnită pe un breadboard, întregul circuit a fost lipit și asamblat pe o placă de prototipare (*perfboard*). Aceasta acționează ca o punte de legătură între unitatea centrală Raspberry Pi 5 și ansamblul de senzori, oferind o stabilitate ridicată din punct de vedere mecanic și reducând riscul de deconectare accidentală a magistrelor de date.

Din punct de vedere al arhitecturii electrice, placa de prototipare a fost structurată pentru a distribui eficient cele două niveluri de tensiune furnizate de portul GPIO al unității Raspberry Pi:

- **Magistrala de 3.3V:** A fost utilizată pentru alimentarea majorității componentelor, asigurând compatibilitatea directă cu pinii de date ai procesorului. La această linie au fost conectate display-ul OLED, senzorul de lumină BH1750, senzorul de umiditate a solului HW-101, convertorul ADS1115 și senzorul de temperatură DS18B20 (împreună cu rezistența sa de pull-up).
- **Magistrala de 5V:** A fost alocată componentelor cu cerințe energetice ridicate. Această magistrală alimentează în paralel senzorul de gaze MQ-135 și senzorul de umiditate AM2302.





### 3.3.1 Arhitectura aplicației locale și mediul de dezvoltare

Software-ul care rulează pe microcalculatorul Raspberry Pi 5 a fost dezvoltat folosind limbajul de programare Python 3. Din punct de vedere arhitectural, aplicația a fost concepută având la bază principiile programării orientate pe obiecte și ale concurenței, pentru a asigura o separare eficientă a responsabilităților hardware de cele de rețea.

Pentru a asigura modularitatea și întreținere a codului, interfațarea cu senzorii fizici nu a fost realizată monolitic, ci una orientată pe componente, stratificată și concurentă. Au fost construite clase **driver** pentru aproape fiecare senzor în parte, cu excepția senzorului AM2302 (pentru acesta s-a folosit biblioteca `adafruit_dht`). Această abordare încapsulează logica de comunicare specifică fiecărui protocol și parametrii de calibrare în module distincte, expunând către scriptul principal exclusiv metode de nivel înalt pentru citirea datelor procesate.

Un aspect critic al arhitecturii software este gestionarea fluxurilor de execuție concurente. Într-un sistem IoT clasic, o întrerupere a rețelei Wi-Fi, o latență a serverului Cloud sau blocajul funcției de monitorizare video poate bloca bucla principală, oprind astfel monitorizarea și controlul termic local. Pentru a preveni acest scenariu, denumit și **Single Point of Failure**, sistemul a fost divizat în trei fire de execuție (thread-uri) sincronizate printr-o structură de date de tip coadă (queue) cu dimensiune maximă limitată, pentru a preveni epuizarea memoriei RAM (descrie în **Secvența de cod 1**):

- **Firul principal:** Responsabil de eșantionarea senzorilor la un interval fix și actualizarea ecranului OLED. Acesta rulează neîntrerupt, indiferent de starea conexiunii la internet, plasând pachetele de date în coada de comunicare.
- **Firul secundar:** Rulează în fundal (*daemon thread*) și are rolul de a prelua asincron datele din coadă pentru a le încărca în baza de date Firebase, evaluând totodată pragurile critice pentru a transmite notificările push către utilizator prin intermediul API-ului Expo.
- **Firul multimedia:** Rulează asincron și e dedicat exclusiv interacțiunii cu camera USB. Acesta preia fotografii la un interval de 5 secunde și le salvează direct în fișierul `/tmp`, asigurând un flux continuu de imagini pentru serverul local fără a încetini timpii de reacție ai componentelor de mediu.

```

1 # queue initialization for the communication between the hardware reading and
   firebase thread
2 telemetry_queue = queue.Queue(maxsize=10) #maxsize = 10 to avoid memory overflow
   if the WiFi connection is down for a long time and the readings are piling up
   in the queue
3
4 # starting the cloud worker thread
5 print("Starting cloud worker thread for Firebase upload and notifications...")
6 cloud_thread = threading.Thread(target=cloud_worker_task, daemon=True)
7 cloud_thread.start()
8
9 # starting the camera thread
10 print("Starting camera worker thread for live view capture...")
11 camera_thread = threading.Thread(target=camera_worker_task, daemon=True)
12 camera_thread.start()
13
14 # main loop
15 # non-blocking put to the telemetry queue, responsible for sending data to the
   cloud worker thread for firebase upload and notification evaluation
16 if not telemetry_queue.full():
17     telemetry_queue.put({'data_dict': data, 'time': current_time})
18 else:
19     print("Warning - your network is slow or down. Dropping telemetry packet to
       prevent overflow")

```

Secvență de Cod 1: Implementarea arhitecturii concurente și mecanismul de evitare a overflow-ului

### 3.3.2 Achiziția datelor și prelucrarea matematică a semnalelor

Procesul de achiziție a datelor în bucla principală a fost proiectat cu un grad ridicat de toleranță la defect. Interogarea fiecărui senzor este izolată prin mecanisme de tratare a excepțiilor (*try-except*), asigurând astfel că o potențială eroare hardware a unei componente nu provoacă oprirea întregului sistem, eroarea fiind tratată corespunzător și afișată în terminal. O provocare software majoră în această etapă a reprezentat-o prelucrarea semnalelor analogice preluate de la convertorul ADS1115 și transformarea lor în mărimi fizice relevante pentru fermă. Aceasta a presupus implementarea unor modele matematice în interiorul claselor **driver**, precum cele pentru **conversia și calibrarea senzorilor de sol și de măsurare a nivelului de CO<sub>2</sub>, calculul concentrației de CO<sub>2</sub> și calculul deficitului de presiune a vaporilor (VPD)**.

**Conversia și calibrarea senzorului de sol:** Deoarece ieșirea senzorului capacitiv de sol reprezintă

o valoare brută de tensiune invers proporțională cu umiditatea, transformarea acesteia într-un procentaj (0%-100%) a necesitat o mapare liniară bazată pe două puncte de calibrare extreme: valoarea în aer uscat (*dry\_val*) și valoarea în saturație de apă (*wet\_val*). Procentajul final este calculat pe baza ecuației:

$$\text{Umiditate sol (\%)} = \frac{\text{dry\_val} - \text{raw\_val}}{\text{dry\_val} - \text{wet\_val}} \cdot 100 \quad (1)$$

Prin fixarea matematică a limitelor superioare și inferioare, s-a prevenit afișarea unor valori eronate (sub 0% sau peste 100%) rezultate din fluctuațiile de tensiune (**descriș în Secvența de cod 2**).

```

1 def get_moisture_percentage(self):
2     """
3     returns the soil moisture level as a percentage (0% = dry, 100% = wet) based on
4     the calibrated dry and wet values
5     """
6     if self.dry_val is None or self.wet_val is None:
7         raise ValueError("Soil moisture sensor is not calibrated. Please run .
8         calibrate() function first")
9
10    raw_value = self.get_raw_value()
11
12    # clamp the raw value between dry and wet values
13    if raw_value > self.dry_val:
14        raw_value = self.dry_val
15    if raw_value < self.wet_val:
16        raw_value = self.wet_val
17
18    # calculating moisture percentage
19    # HW-101 is a capacitive sensor - higher ADC values - drier soil, lower ADC
20    values - wetter soil
21    # formula: moisture% = ((dry - raw) / (dry - wet)) * 100
22    moisture_percentage = (self.dry_val - raw_value) / (self.dry_val - self.wet_val)
23        * 100.0
24    return round(moisture_percentage, 1)

```

Secvența de Cod 2: Implementarea limitării superioare/inferioare și maparea liniară a umidității

**Calibrarea senzorului MQ-135 și calculul concentrației de CO<sub>2</sub>:** Determinarea precisă a concentrației de CO<sub>2</sub> în PPM necesită un proces bazat pe două etape fundamentale: calibrarea senzorului MQ-135 în aer curat și calculul exponențial continuu. În faza de calibrare, se calculează inițial rezistența curentă a senzorului (*R<sub>s</sub>*) în raport cu o rezistență de sarcină (*RL*) de 1 kΩ, atașată pe modulul

senzorului. Transformarea tensiunii analogice citite de către convertorul ADC în rezistență se realizează folosind ecuația divizorului de tensiune:

$$R_s = R_L \cdot \frac{V_{in} - V_{out}}{V_{out}} \quad (2)$$

Pentru a diminua impactul zgomotului electromagnetic specific semnalelor analogice, s-a implementat o tehnică de filtrare software de tip Low-Pass. Algoritmul de calibrare extrage 20 de eșantioane succesive la intervale de 0.1 secunde, calculând media aritmetică a rezistenței. Folosind această medie și o concentrație de referință (420 PPM pentru aerul exterior), se izolează matematic și se stochează rezistența de bază a senzorului  $R_o$ . Odată stabilită constanta  $R_o$ , logica de calcul a concentrației curente se bazează pe modelul matematic al curbei de sensibilitate a senzorului MQ-135 [55] [56]. Concentrația de  $CO_2$  este determinată printr-o funcție de putere, unde factorul de scalare (a) și exponentul (b) sunt constante specifice extrase din foaia de catalog [55] [56]:

$$PPM = a \cdot \left( \frac{R_s}{R_o} \right)^b \quad (3)$$

Această implementare, incluzând protecția împotriva diviziunii cu zero și validarea limitelor hardware admise pentru raportul  $R_s/R_o$ , este prezentată în **secvența de cod 3**.



```

1 def get_ppm(self):
2     """
3     returns the ppm (parts per million) based on the sensor resistance (Rs) and the
4     calibrated Ro value
5     Formula: ppm = a*(Rs/Ro)^b, a - MQ135_SCALINGFACTOR, b - MQ135_EXPONENT
6     """
7     resistance = self.get_resistance()
8     # avoid invalid resistance values
9     if resistance <=0:
10         return 0.0
11     #calculate the ratio between Rs and Ro
12     ratio = resistance / self.ro
13     #calculate PPM based on the formula
14     if ratio > self.MQ135_MIN_RSRO_RATIO and ratio < self.MQ135_MAX_RSRO_RATIO:
15         ppm = self.MQ135_SCALINGFACTOR * math.pow(ratio, self.MQ135_EXPONENT)
16     else:
17         ppm = 0.0 #out of valid range
18     return ppm

```

Secvență de Cod 3: Implementarea algoritmului de calcul exponențial pentru concentrația de CO<sub>2</sub>

**Calculul deficitului de presiune a vaporilor (VPD):** Dincolo de citirea valorilor achiziționate, sistemul implementează calculul VPD-ului, un parametru vital în biologia ciupercilor, ce indică rata de transpirație a ciupercilor. Algoritmul (**prezentat în secvența de cod 4**) utilizează formula Tetens [57] pentru a determina **presiunea vaporilor de saturație (SVP)** în funcție de temperatura aerului în grade Celsius:

$$SVP = 0.61078 \cdot e^{\frac{17.27 \cdot T}{T+237.3}} \quad (4)$$

Ulterior, prin corelarea SVP cu umiditatea relativă, sistemul deduce valoarea finală a VPD-ului (exprimată în kPa), oferind cultivatorului un indicator de performanță superior unei simple citiri a temperaturii.

```

1 # function for VPD calculation (vapor pressure deficit) - important parameter for
  mushroom growth, calculated based on temperature and air humidity readings
2 def calculate_vpd(temperature_c, air_humidity):
3     try:
4         # saturation vapor pressure calculation using Tetens formula
5         # SVP = 0.61078 * e^((17.27 * T) / (T + 237.3)), where T is the temp in
          celsius
6         svp = 0.61078 * math.exp((17.27 * temperature_c) / (temperature_c + 237.3)) #
          measured in kPa
7
8         # actual vapor pressure calculation based on svp and air humidity
9         avp = svp * (air_humidity / 100.0) # measured in kPa
10        # vpd = svp - avp
11        return round(svp - avp, 2) # return vpd rounded to 2 decimal
12    except Exception as e:
13        print(f"Error calculating VPD: {e}")
14        return None

```

Secvență de Cod 4: Transpunerea ecuației Tetens în Python pentru determinarea VPD-ului

### 3.3.3 Sistemul de captură vizuală

Pentru a adăuga o componentă de monitorizare vizuală, arhitectura software de pe unitatea de calcul a fost extinsă cu un sistem dedicat capturii fototografice asincrone. Această funcționalitate este integrată prin lansarea unui fir de execuție secundar, complet independent de procesul de interogare a senzorilor fizici.

Firul de execuție destinat gestionării camerei web rulează într-o buclă infinită, fiind programat să realizeze o poză din interiorul incintei la un interval de 5 secunde. În faza de proiectare a acestui modul, a fost acordată atenția asupra gestionării resurselor fizice de stocare ale microcalculatorului. Salvarea constantă a unei imagini pe cardul microSD la fiecare 5 secunde ar fi generat un volum extrem de mare de operațiuni de scriere și ștergere ciclică, conducând la uzura și la defectarea fizică a stocării într-un timp foarte scurt.

Pentru a elimina acest risc, s-a recurs la o tehnică de optimizare la nivelul sistemului de operare Linux, mai exact salvarea cadrelor capturate direct în partiția temporară /tmp. În sistemele bazate pe nucleul Linux, acest director este mapat direct în memoria RAM. Această abordare arhitecturală oferă viteze de citire și scriere superioare mediilor de stocare flash, garantează o latență minimă la procesarea cadrelor și elimină în totalitate degradarea hardware a cardului de memorie, funcționând exclusiv în memoria volatilă a Raspberry Pi 5.

Odată ce imaginea este salvată în memoria RAM, aceasta trebuie să fie accesibilă la nivel de rețea pentru a putea fi preluată de aplicația mobilă. În acest scop, a fost instanțiat un server web HTTP, configurat să asculte cererile primite pe portul 8000. Sarcina principală a acestui server este să intercepteze cererile de tip **GET** venite din rețea și de a livra instantaneu cea mai recentă fotografie stocată în directorul /tmp. Deoarece microcalculatorul funcționează într-o rețea locală privată, serverul HTTP local nu poate fi interogată direct de pe internet de către aplicația mobilă. Pentru a rezolva această problemă, am folosit platforma Ngrok. Prin aceasta, putem deschide un tunel securizat la nivel de transport către serverele externe Ngrok, generând un endpoint public unic protejat prin criptare HTTPS. Acest URL securizat redirecționează solicitările aplicației mobile direct către portul 8000 al plăcii, oferind utilizatorului un flux vizual actualizat din orice locație.

### 3.3.4 Schema logică a fluxului principal de execuție

Pentru a oferi o viziune de ansamblu și a prezenta sumar conceptele arhitecturale și algoritmice detaliate în secțiunile anterioare, a fost elaborată diagrama logică de flux a programului principal, ilustrată în **Figura 11**. Această reprezentare vizuală expune ciclul de viață al firului de execuție principal, responsabil pentru componenta de Edge Computing a sistemului.

Diagrama evidențiază monitorizarea datelor și separarea funcțională a etapelor. Procesul începe cu faza de inițializare a componentelor hardware și a magistralelor de comunicație, urmată de instanțierea cozii de mesaje și lansarea în fundal a firelor de execuție responsabile atât pentru stocarea datelor în Cloud și transmiterea de notificări push, cât și pentru realizarea capturilor foto la un interval de 5 secunde.

Ulterior, algoritmul intră în bucla infinită de monitorizare, care este responsabilă de **achiziția și prelucrarea datelor și distribuția datelor**.

## 3.4 Arhitectura platformei Cloud

Pentru a asigura monitorizarea de la distanță a fermei și a permite modificarea dinamică a parametrilor de control fără intervenție fizică asupra unității centrale, sistemul a fost integrat cu ecosistemul Firebase. Această decizie facilitează stocarea datelor într-o bază de date NoSQL în timp real, oferind un grad ridicat de scalabilitate și flexibilitate în gestionarea formatelor de date de tip JSON.

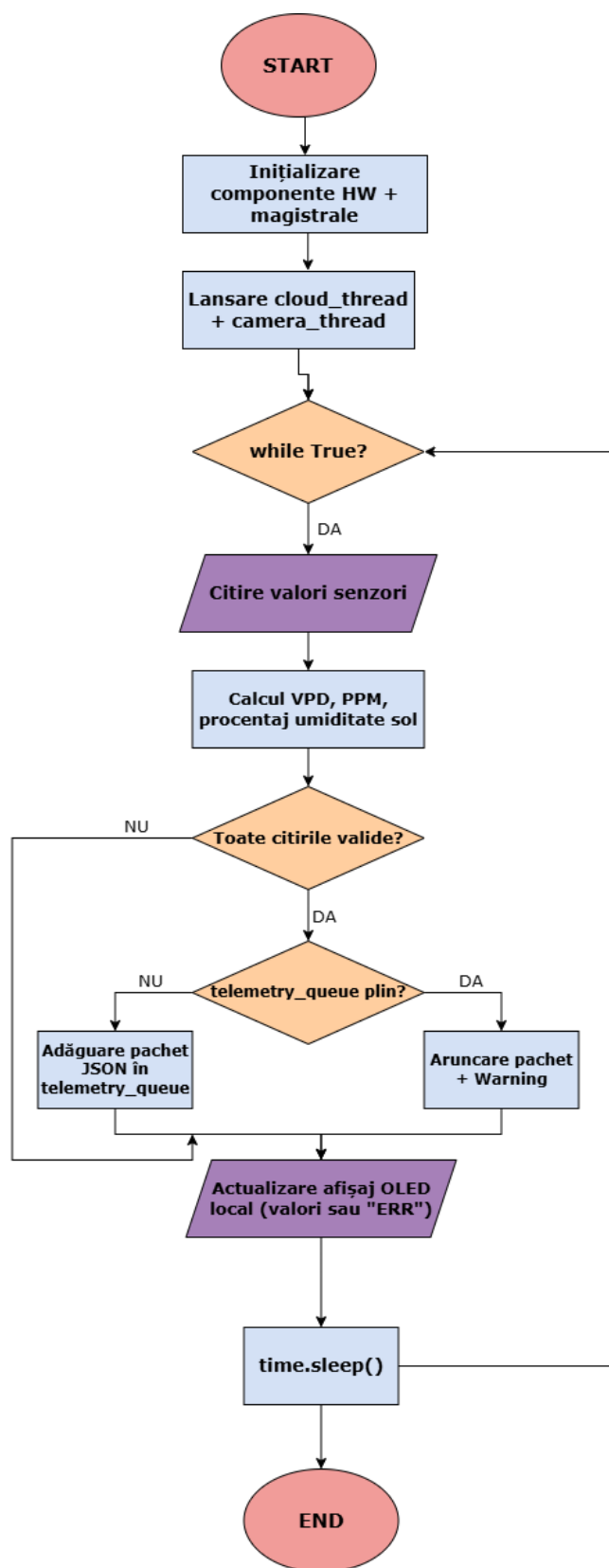


Figura 11: Schema logică a fluxului de execuție

### 3.4.1 Dezvoltarea modului de comunicare cu baza de date

Pentru a respecta principiile programării orientate pe obiecte și a menține o separare între logica de control și tranzacțiile de rețea, întreaga comunicare cu Firebase a fost încapsulată în clasa **FirebaseManager**. Acest modul acționează ca un strat de abstractizare între scriptul principal și baza de date NoSQL.

S-a optat pentru utilizarea ecosistemului Firebase în favoarea protocolului tradițional MQTT, deoarece Firebase elimină necesitatea configurării unui broker de mesaje intermediar, oferind simultan stocare persistentă și sincronizare bidirecțională prin WebSockets, funcționalități necesare pentru conectarea eficientă a interfeței client.

Structura acestui modul a fost proiectată pentru a rezolva patru cerințe principale ale sistemului IoT:

**1. Securitatea credențialelor și inițializarea:** Pentru a preveni expunerea cheilor de acces în codul sursă, instanțierea aplicației Firebase utilizează variabile de mediu preluate dintr-un fișier local securizat (`.env`), prin intermediul bibliotecii `dotenv`. Certificatul de autentificare este reconstruit sub forma unui dicționar JSON, oferind o conexiune criptată și autorizată la nodul rădăcină al proiectului (`proiect-licenta`).

**2. Structura de stocare a datelor:** Pentru a optimiza costurile de stocare și lățimea de bandă, funcția `upload_data()`, descrisă în **Secvența de Cod 5**, folosește metode de scriere asimetrică și tehnici de sub-eșantionare:

- **Date în timp real (`current_readings`):** Pachetele de date sunt suprascrise la fiecare iterație a buclei principale la un interval de 3 secunde, folosind comanda `set()`. Această abordare permite interfeței mobile să afișeze instant starea senzorilor, fără a acumula un istoric inutil. Intervalul de 3 secunde oferă o latență insesizabilă pentru utilizatorul aplicației mobile, evitând în același timp blocajele de rețea.
- **Date istorice (`past_readings`):** Pentru a permite analiza pe termen lung a fluctuațiilor de valori ai fermei, un eșantion de date este salvat permanent la fiecare 5 minute folosind comanda `push()`. Dacă sistemul ar rula constant pentru o zi întreagă, ar genera 288 de eșantioane pe zi, oferind o rezoluție suficientă pentru analiza fluctuațiilor microclimatului.

```

1 def upload_data(self, data_dictionary):
2     """
3     param data_dictionary: a dictionary containing the sensor readings to be
4         uploaded to firebase
5     """
6     try:
7         # current readings node - updating every 3 seconds (SAMPLE_INTERVAL) in main
8         loop
9         self.root_ref.child('current_readings').set(data_dictionary) # overwriting the
10            current data with new data
11        print("[CL-T] Current readings updated in Firebase")
12        current_time = data_dictionary['last_updated'] / 1000 # convert timestamp back
13            to seconds for easier time calculations
14
15        # past readings node - updating every 5 minutes
16        if current_time - self.last_upload_time >= self.upload_interval:
17            self.root_ref.child('past_readings').push(data_dictionary) # pushing data to
18                past_readings, creating a new entry with a unique key on timestamp
19            self.last_upload_time = current_time
20            print("Past readings uploaded to Firebase")
21    except Exception as e:
22        print(f"Firebase Sync Error: {e}")

```

Secvență de Cod 5: Implementarea mecanismului asincron de scriere pentru date în timp real și istoric

**3. Sincronizarea bidirecțională și actualizarea pragurilor de la distanță:** O funcționalitate importantă adusă de acest modul este capacitatea de a modifica pragurile de control termic fără a reporni sistemul. În constructorul clasei, a fost atașat un proces de ascultare în fundal (*background listener*) pe nodul `farm_settings`. Metoda asincronă `_settings_listener(event)`, descrisă în **Secvența de Cod 6**, detectează orice modificare efectuată de utilizator din aplicația mobilă și actualizează dinamic dicționarul local `farm_thresholds`.

```

1 def _settings_listener(self, event):
2     """
3     runs automatically in the background in response to changes in the '
4         farm_settings' node in firebase
5     """
6     incoming_data = event.data
7     if incoming_data:
8         print("Mobile app update received!")

```

```

9     if event.path == '/': # if the entire farm_settings node is updated
10         self.farm_thresholds.update(incoming_data) # update the local
farm_thresholds with the new values from firebase
11     else:
12         key = event.path.replace('/', '') # get the key of the updated string
13         if key in self.farm_thresholds:
14             if isinstance(incoming_data, dict):
15                 self.farm_thresholds[key].update(incoming_data) # update the specific
parameter thresholds with the new values from firebase
16             else:
17                 self.farm_thresholds[key] = incoming_data # if the incoming data is not
a dictionary, update the entire parameter thresholds with the new value from
firebase
18
19     print("New thresholds successfully applied without rebooting!")

```

Secvență de Cod 6: Implementarea \_settings\_listener pentru actualizarea pragurilor în timp real

**4. Optimizarea extragerilor de date și gestionarea alertelor:** Pentru a eficientiza trimiterea notificărilor push, token-ul de identificare al dispozitivului mobil este descărcat la fiecare alertă. Acesta este preluat o singură dată la inițializarea sistemului prin funcția `_fetch_initial_push_token()` și stocat în memoria cache. De asemenea, pentru a preveni transmiterea unui număr mare de alerte, funcția `register_alert()`, descrisă în **Secvența de Cod 7**, implementează o perioadă de pauză de 15 minute per parametru monitorizat. Sistemul scrie în log-ul de alerte din baza de date doar dacă acest interval a fost depășit.

```

1 def register_alert(self, message, alert_type, parameter):
2     """
3     pushes an alert to firebase if the cooldown period (15 min) has passed
4     """
5     now = time.time()
6
7     #check if the cooldown period has passed for this parameter
8     if (now - self.last_alert_times.get(parameter, 0)) > self.alert_cooldown:
9         alert_data = {
10             "timestamp": int(now*1000),
11             "message": message,
12             "type": alert_type,
13             "parameter": parameter
14         }
15     try:

```

```

16     self.root_ref.child('alerts').push(alert_data) # push the alert to firebase,
    creating a new entry with a unique key on timestamp
17     self.last_alert_times[parameter] = now
18     print(f"Alert sent to Firebase: {message}")
19     # function telling proiect_licenta.py to trigger the push notification to
    the mobile app through the Expo push notification service, using the push token
    stored in firebase
20     return True
21 except Exception as e:
22     print(f"Failed to send alert to Firebase: {e}")
23     return False # if the alert failed to send to firebase, we return False so
    that proiect_licenta.py does not trigger the push notification
24     return False # if the cooldown period has not passed, we return False so that
    proiect_licenta.py does not trigger the push notification

```

Secvență de Cod 7: Implementarea funcției de trimitere a alertelor la interval de 15 minute

### 3.4.2 Gestionarea fluxului de rețea

Odată ce clasa `FirebaseManager` a fost definită, integrarea acesteia în logica microcalculatorului s-a realizat ținând cont de latențele rețelelor wireless. Orice operațiune de tip I/O pe rețea, cum ar fi o tranzacție către serverele Google, blochează firul curent de execuție până la primirea pachetului. Dacă această rutină de sincronizare ar fi fost plasată în bucla principală de control, o eventuală instabilitate a rețelei Wi-Fi ar fi provocat înghețarea sistemului, blocând complet eșantionarea senzorilor la intervalele de timp prestabilite.

Pentru rezolvarea acestei probleme, s-a evaluat inițial utilizarea programării asincrone (prin bibliotecă `asyncio`). Totuși, această abordare nu a fost considerată compatibilă din punct de vedere al arhitecturii sistemului. Un model asincron bazat pe o buclă de evenimente necesită apeluri non-blocante. În arhitectura sistemului prezentat, atât driverele senzorilor, cât și metodele SDK-ului `Firebase`, sunt funcții blocante. Implementarea unui model asincron în acest context ar fi anulat avantajele concurenței, ducând la blocarea repetată a execuției programului.

Astfel, s-a folosit o arhitectură bazată pe fire de execuție multiple și paradigma `Producer-Consumer`. Pentru a asigura o separare completă și a garanta funcționarea sistemului de calcul local, s-a implementat funcția `cloud_worker_task()`, rulată independent într-un thread secundar de tip `daemon` (prezentată în **Secvența de Cod 8**). Firul principal acționează ca un producător de date, în timp ce noua componentă acționează ca un consumator direct pentru coada `telemetry_queue`.

Funcționarea acestui thread este complet decuplată față de hardware și îndeplinește trei funcțion-



alități principale:

- **Transmiterea datelor:** Firul de execuție așteaptă starea de disponibilitate a datelor, apelând metoda blocantă la nivel de thread `get()`. Imediat ce un pachet JSON este plasat în coadă de către firul principal, acesta este extras și transmis către Firebase prin apelarea metodei `upload_data()`.
- **Preluarea setărilor:** După finalizarea etapei de upload, sistemul interoghează instanța managerului Firebase pentru a prelua cele mai recente praguri de control configurate de utilizator din aplicația mobilă. Aceste limite sunt transferate într-o variabilă globală (`active_thresholds`), permițând thread-ului principal să citească noile valori pentru evaluarea limitelor de alarmare, evitând blocajele de tip Race Conditions.
- **Evaluarea alertelor:** Pentru a nu încărca procesorul inutil, evaluarea condițiilor de declanșare a alarmelor este calculată exclusiv în acest thread secundar, folosind datele recent extrase din coadă. Sistemul verifică fluctuațiile senzorilor și decide dacă este necesară inițierea unui flux de alertă.

```

1 active_thresholds = None
2
3 # function to run the cloud uploader and notification trigger in a separate thread
  , to avoid blocking the main thread in case of WiFi connectivity issues or
  firebase errors
4 def cloud_worker_task():
5     global active_thresholds # used for storing the latest thresholds fetched from
      firebase
6     while True:
7         try:
8             payload = telemetry_queue.get() # blocking call, waits for new data from the
              main thread
9             data = payload['data_dict']
10            current_time = payload['time']
11
12            #uploading data to firebase
13            firebase_manager.upload_data(data)
14            print("[CL-T] Data uploaded to Firebase")
15
16            #getting the live thresholds and update global variable 'active_thresholds'
17            fetched_thresholds = firebase_manager.get_thresholds()
18            if fetched_thresholds:

```

```

19     active_thresholds = fetched_thresholds
20
21     #evaluate sensors for push notifications
22     if active_thresholds:
23         evaluate_sensor(data.get('temperature_c'), 'temperature_c', 'Temperature',
24                             '°C', current_time, active_thresholds)
25         evaluate_sensor(data.get('humidity_percent'), 'humidity_percent', '
Humidity', '%', current_time, active_thresholds)
26         # evaluate the rest of the sensors too
27         telemetry_queue.task_done() # mark the task as done after processing
28     except Exception as e:
29         print(f"Error in cloud worker thread: {e}")

```

Secvență de Cod 8: Implementarea thread-ului secundar pentru procesarea asincronă a pachetelor de date

### 3.4.3 Sistemul de alerte și integrarea notificărilor push

Un obiectiv principal al aplicației IoT este informarea constantă a utilizatorului în cazul unor fluc-tuații critice ale fermei, eliminând necesitatea monitorizării manuale a interfeței grafice. Pentru a atinge acest obiectiv, sistemul implementează un modul decizional de evaluare a parametrilor (de-scris în **Secvența de Cod 9**), integrat direct în fluxul thread-ului secundar de execuție, responsabil de transmiterea datelor în rețea.

Logica de avertizare a fost proiectată pe două niveluri de severitate, folosind o funcție de evaluare pentru fiecare parametru în parte:

- **Warning:** Pentru a permite intervenția preventivă a cultivatorului, algoritmul calculează un buffer reprezentând 15% din valorile intervalului optim. Dacă un parametru se apropie de pragurile limită, intrând în acest buffer, sistemul declanșează o alertă de avertizare a utilizatorului.
- **Critical:** Declanșată în momentul în care valoarea parametrului a depășit limitele absolute de siguranță, indicând un risc major pentru cultura fungică.

```

1 # threshold evaluator - calculates buffer zones and triggers appropriate alerts
2 def evaluate_sensor(value, param_key, display_name, unit, current_time, thresholds
3     ):
4     if value is None:
5         return #skip evaluation if sensor failed to read
6     critical_low = thresholds[param_key]["min"]
7     critical_high = thresholds[param_key]["max"]

```

```

8  # 15% warning buffer inside the safe zone
9  range_span = critical_high - critical_low
10 buffer = range_span * 0.15
11 warning_low = critical_low + buffer
12 warning_high = critical_high - buffer
13
14 # evaluate critical conditions first
15 if value <= critical_low or value >=critical_high:
16     trigger_alarm(
17         param_key,
18         f"Critical {display_name}",
19         f"{display_name} is {value:.1f}{unit} (Outside {critical_low} - {
20             critical_high}{unit})",
21         "critical",
22     )
23 # check warning buffer second
24 elif value <= warning_low or value >= warning_high:
25     trigger_alarm(
26         param_key,
27         f"{display_name} Warning",
28         f"{display_name} is {value:.1f}{unit} (Approaching limits of {critical_low} -
29             {critical_high}{unit})",
30         "warning",
31     )

```

Secvență de Cod 9: Algoritmul de calcul al buffer-elor și evaluarea valorilor

Odată ce neregularitatea este detectată de funcția de evaluare, execuția este delegată mecanismului ce se ocupă de transmiterea notificărilor. Pentru a asigura livrarea promptă a mesajelor către aplicația mobilă, s-a folosit serviciul Expo Push Notifications.

Funcția responsabilă de declanșarea alarmei (prezentată în **Secvența de Cod 10**) verifică inițial respectarea perioadei de pauză interogând instanța managerului Firebase. Dacă alerta este validată ca fiind legitimă, sistemul preia din memoria cache token-ul unic de identificare al dispozitivului mobil. Ulterior, se construiește un pachet de date JSON care respectă specificațiile API-ului Expo, conținând titlul, mesajul și prioritatea alertei. Transmiterea se realizează printr-un apel HTTP asincron de tip POST către punctul terminal al serviciului de notificări.

```

1  # checks if the cooldown period has passed since the last alert for the given
    sensor type, and if so, sends a push notification to the mobile app using the
    Expo push notification service
2  def trigger_alarm(sensor_type, title, message, alert_type):

```

```
3
4 is_new_alert = firebase_manager.register_alert(message, alert_type, sensor_type)
    # write to the alert log in firebase and check if the cooldown period has
    # passed to trigger a new alert
5
6 if is_new_alert:
7     print(f"Sending push notification for {sensor_type} alert: {title}")
8     token = firebase_manager.get_push_token() # get the push token from firebase
9
10    # if the token is available, send the push notification to the mobile app
11    if token:
12        # construct the payload for the Expo push notification API
13        headers = {
14            # headers parameters definition
15        }
16        payload = {
17            # payload of the push notifications defined here accordingly
18        }
19
20        # send the POST req to the Expo push notification endpoint with the payload
21        try:
22            requests.post(EXPO_PUSH_URL, headers=headers, json=payload, timeout=5)
23            print("Push notification sent successfully\n")
24        except Exception as e:
25            print(f"Failed to send push notification: {e}")
26    else:
27        print("No push token available, cannot send notification")
```

Secvență de Cod 10: Integrarea API-ului Expo pentru transmiterea cererilor HTTP de tip Push Notification

### 3.5 Dezvoltarea aplicației mobile pentru monitorizare și control

Pentru a oferi cultivatorului o interfață portabilă și capabilă să afișeze datele în timp real, a fost dezvoltată o aplicație mobilă. Aceasta închide bucla sistemului IoT, transformând datele stocate în baza de date Cloud în informații vizuale și permițând modificarea de la distanță a parametrilor de funcționare.

### 3.5.1 Arhitectura aplicației client și mediul de dezvoltare

Aplicația mobilă a fost proiectată utilizând framework-ul **React Native**, care permite dezvoltarea sistemelor cross-platform dintr-o singură bază de cod. Pentru a asigura o verificare statică a erorilor în timpul dezvoltării, proiectul a fost scris în **TypeScript**, adăugând un strat de siguranță în ceea ce privește tipurile de date, comparativ cu limbajul JavaScript, care nu oferă această funcționalitate.

Gestionarea aplicației, accesul la API-urile native ale dispozitivului și sistemul de navigare au fost facilitate de platforma **Expo**. Mai exact, rutarea între ecranele aplicației utilizează Expo Router, un sistem modern de navigare bazat pe sistemul de fișiere (*file-based routing*), care structurează logic componentele vizuale în fișiere dedicate.

Din punct de vedere al ingineriei software, aplicația respectă principiul separării responsabilităților (*Separation of Concerns*). Logica de interogare a bazei de date și prelucrarea datelor au fost extrase din componentele vizuale și încapsulate în funcții personalizate de tip *Custom Hooks*. Preferințele utilizatorului, precum conversia unităților de măsură a temperaturii (Celsius/Fahrenheit) sau tema vizuală (Dark/Light mode) sunt transmise global în întregul arbore de componente prin intermediul unui *Context API* (`PreferencesProvider`).

Deoarece aplicația controlează un sistem fizic, care poate să fie manipulat inclusiv de persoane neautorizate, implementarea unui mecanism de securitate a reprezentat o necesitate. Arhitectura de acces este împărțită pe două niveluri:

- **Autentificarea Cloud:** Accesul inițial este condiționat de validarea credențialelor (adresă de e-mail și parolă) prin serviciul Firebase Authentication. Formularul de înregistrare (`auth.tsx`) implementează validări de securitate stricte prin expresii regulate, impunând o complexitate ridicată a parolei. Logica de rutare din componenta principală (`_layout.tsx`) acționează ca un gardian, redirectionând automat utilizatorii neautentificați către ecranul de Login și blocând accesul la datele senzorilor.
- **Autentificarea biometrică locală:** Pentru a proteja datele sensibile odată ce utilizatorul este conectat la aplicație, s-a folosit API-ul `expo-local-authentication`. Odată ce utilizatorul este conectat în contul Firebase, la fiecare redeschidere a aplicației din fundal, interfața se blochează automat, solicitând validarea identității prin senzorul biometric al telefonului (amprentă sau recunoaștere facială) înainte de a permite accesul la aplicație. Funcționalitatea este descrisă în **Secvența de Cod 11**.

```
1 // trigger biometric auth again when the app comes back from the background
2 useEffect(() => {
```

```

3  const subscription = AppState.addListener("change", nextAppState => {
4    // if the app comes back to the foreground and the user is logged in, we want
    // to trigger biometric authentication again
5    if(appState.current.match(/inactive|background/) && nextAppState === "active")
6    {
7      if (user) {
8        setIsUnlocked(false); // lock the app when it goes to the background
9        authenticateUser(); // trigger biometric authentication when the app comes
        back from the background
10     }
11   }
12   appState.current = nextAppState;
13 });
14 // cleanup the event listener
15 return () => {
16   subscription.remove();
17 }, [user]);

```

Secvență de Cod 11: Implementarea securității biometrice la reactivarea aplicației din fundal

Pentru dezvoltarea și organizarea aplicației mobile, s-a implementat o arhitectură software structurată pe straturi ierarhice. Această metodă de proiectare asigură respectarea principiului de separare a responsabilităților, despărțind complet interfața grafică utilizator de logica computațională de gestiune a stărilor și de protocoalele de comunicație hardware cu infrastructura Cloud. Prin această abordare, codul sursă poate să fie menținut foarte ușor, componentele pot să fie testate separat, iar sistemul poate să fie extins foarte ușor cu noi funcționalități, abordarea oferind o scalabilitate ridicată.

Arhitectura prezentată în **Figura 12** poate fi împărțită în trei straturi software interne, conectate cu două entități externe de stocare și rețea:

### 1. Stratul de interfață grafică utilizator (UI)

Acesta reprezintă nivelul superior al stivei software, responsabil în mod exclusiv de afișarea elementelor vizuale, maparea componentelor grafice și de interceptarea acțiunilor utilizatorului. Punctul de control ierarhic și de gestiune a rutelor este deservit de componenta principală (`_layout.tsx`), care evaluează permisiunile de acces și securitatea sesiunii. În cadrul acestui strat, ecranele sunt organizate în două feluri:

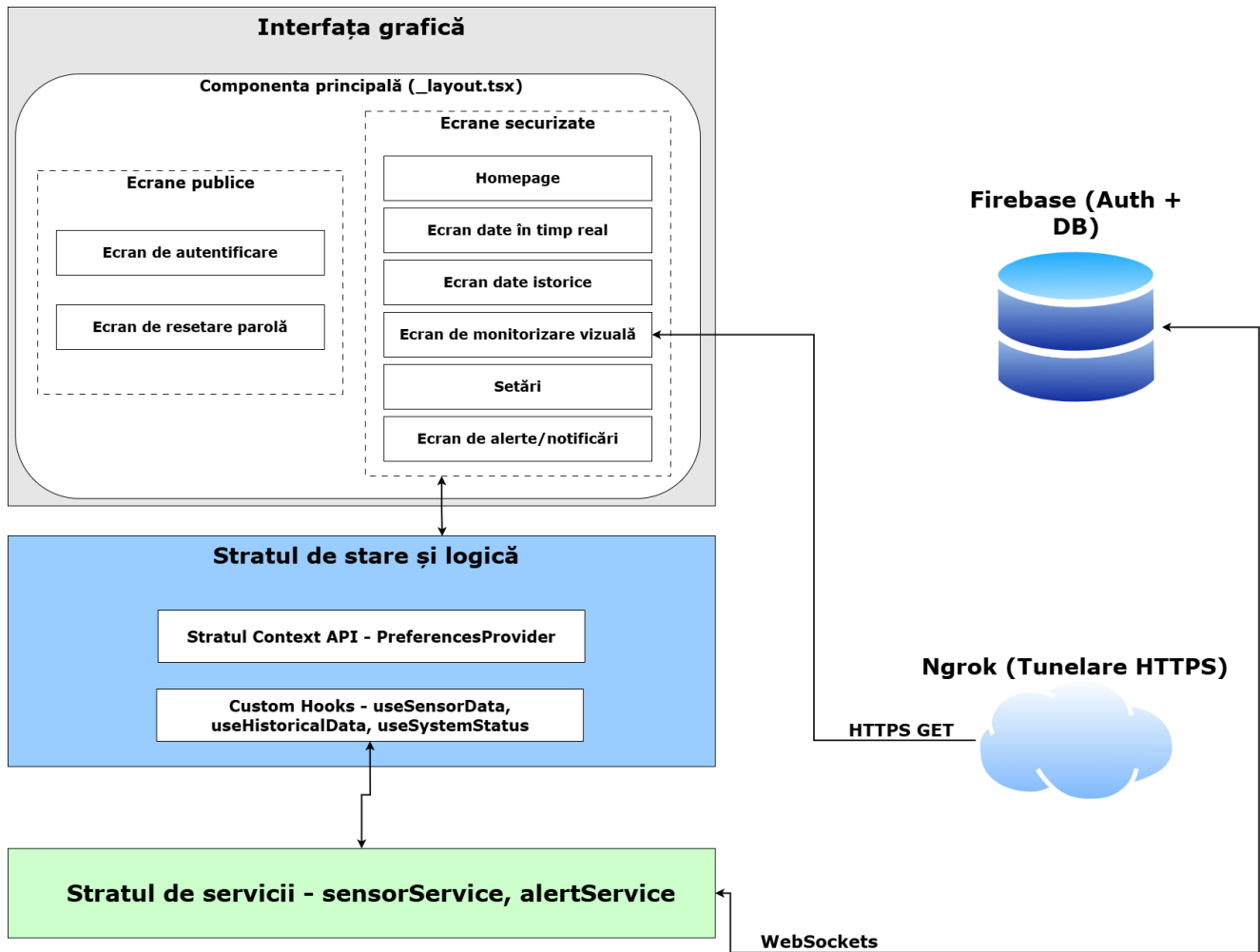


Figura 12: Arhitectura aplicației mobile

- **Ecrane publice:** Responsabile de interfața accesibilă înaintea procesului de validare a utilizatorului, cuprinzând ecranul de autentificare și cel dedicat resetării parolei contului.
- **Ecrane securizate:** Reprezintă spațiul de lucru al aplicației după autentificarea cu succes. Acest bloc integrează ecranul principal, cel pentru vizualizarea datelor în timp real, pentru analiza grafică a datelor istorice, ecranul de setări și cel de alerte. În această structură este integrat și ecranul de monitorizare vizuală, responsabil de afișarea cadrelor preluate de la distanță.

## 2. Stratul de stare și logică

Acest strat funcționează ca nucleul decizional al frontend-ului, realizând transformarea datelor brute primite prin rețea în stări interpretabile de către interfața grafică. Acesta elimină dependența directă a ecranelor de sursele de date prin două componente importante:

- **Context API - PreferencesProvider:** Rulează ca un container de stare global, distribuind preferințele utilizatorului, parametrii de stil și starea curentă a sesiunii de autentificare către întregul arbore de componente vizuale, eliminând fenomenul de transmitere repetitivă a proprietăților prin componente intermediare.
- **Stratul Custom Hooks:** Încapsulează logica nativă React sub forma unor funcții software specializate (`useSensorData`, `useHistoricalData`, `useSystemStatus`). Acestea gestionează asincron ciclurile de viață ale datelor, reîmprospătarea periodică a datelor și subscrierile la modificările de stare.

## 3. Stratul de servicii

Acest strat reprezintă baza software-ului intern al aplicației, ce are rolul de a abstractiza complet conexiunile de rețea prin intermediul modulelor specializate (`sensorService`, `alertService`). Acest strat preia apelurile logice primite de la nivelul superior, construiește structura cererilor HTTP/WebSocket și procesează răspunsurile, separând straturile de UI și de logică de detaliile de implementare ale API-urilor externe.



### 3.5.2 Interfața de monitorizare în timp real și evaluarea stării sistemului

Odată ce utilizatorul a trecut de barierele de securitate și a accesat grupul de ecrane protejate, funcția principală a clientului mobil este de a afișa parametrii în timp real ai microclimatului din ferma de ciuperci. Această cerință a fost implementată prin intermediul ecranului de date live (`live_data.tsx`) și al sistemului de navigație către ecranul ce conține log-ul de evenimente.

Spre deosebire de aplicațiile web clasice care necesită reîmprospătarea manuală a paginii sau interogări repetate, interfața aplicației utilizează un model de arhitectură reactivă (*Push-API Architecture*). Prin intermediul hook-ului personalizat `useSensorData`, ecranul se abonează asincron la nodul de date curente din Firebase. Orice actualizare efectuată de unitatea Raspberry Pi declanșează automat un eveniment care populează obiectul `sensorData`, actualizând componentele vizuale individuale cu o latență minimă. Astfel, cultivatorul vizualizează fluctuațiile de temperatură, umiditate sau nivel de  $\text{CO}_2$  exact în momentul producerii evenimentului.

O provocare în sistemele IoT o reprezintă deconectarea silențioasă, o situație în care microcontrollerul cedează sau rețeaua Wi-Fi locală pică, iar interfața continuă să afișeze ultimele date cunoscute, dând falsa impresie că parametrii biologici sunt în limite normale. Pentru a rezolva acest risc, pe ecranul principal a fost integrat un indicator dinamic de stare (*System Status Banner*). Logica matematică a fost extrasă și încapsulată în modulul `useSystemStatus.ts` (prezentat în **Secvența de Cod 12**), pentru a respecta principiul responsabilității unice. Aplicația menține un ceas intern asincron care se actualizează la fiecare 10 secunde. Acest timp curent este comparat permanent cu marcajul temporal asociat ultimului pachet primit de la baza de date. Dacă diferența de timp calculată depășește pragul maxim de toleranță de 5 minute (`OFFLINE_TIMEOUT_MS`), sistemul deduce că nodul fizic a încetat să mai transmită date. Interfața își schimbă automat aspectul și starea din **"System Online"** în **"System Offline"**, indicând simultan timpul scurs de la ultima sincronizare reușită.

```

1 // hook to calculate system status (online/offline) based on the last updated
  timestamp of the sensor data and the current time
2 export const useSystemStatus = (lastUpdated: number | null) => {
3   const [now, setNow] = useState(Date.now());
4   const [status, setStatus] = useState<SystemStatus>({
5     isOnline: false,
6     timeAgoText: "Waiting for data...",
7   });
8
9   // effect to update the current time every 10 seconds so "last updated" can be
    calculated in real-time
10  useEffect(() => {

```

```

11   const interval = setInterval(() => {
12     setNow(Date.now());
13   }, 10000); // update every 10 seconds
14   return () => clearInterval(interval);
15 }, []);
16
17 // calculate status based on lastUpdated and current time
18 useEffect(() => {
19   if (!lastUpdated) {
20     setStatus({
21       isOnline: false,
22       timeAgoText: "Waiting for data...",
23     });
24     return;
25   }
26   const isOnline = now - lastUpdated <= THRESHOLDS.OFFLINE_TIMEOUT_MS;
27   const diffInSeconds = Math.max(0, Math.floor((now - lastUpdated) / 1000));
28   let timeAgoText = "Waiting for data...";
29   if (diffInSeconds < 60) {
30     timeAgoText = `Updated ${diffInSeconds} s ago`;
31   }
32   // same formulas for diffInMins, diffInHours and diffInDays, based on seconds
33   setStatus({ isOnline, timeAgoText });
34 }, [lastUpdated, now]);
35 return status;
36 };

```

Secvență de Cod 12: Implementarea mecanismului matematic pentru evaluarea stării conexiunii IoT

### 3.5.3 Vizualizarea analitică a datelor istorice și modulul de export

În cultivarea automatizată, analiza datelor în timp real este insuficientă pentru detectarea modificărilor lente a parametrilor. Pentru a oferi cultivatorului o perspectivă mai largă asupra evoluției culturii, a fost dezvoltat ecranul de analiză a datelor istorice (`history_chart.tsx`). Acest modul preia seturile mari de date stocate în nodul `past_readings` din Firebase și le transpune în reprezentări grafice interactive.

Vizualizarea datelor utilizează biblioteca `react-native-chart-kit`, configurată pentru a genera linii fluide pe graficele fiecărui parametru măsurat. Ecranul mapează independent istoricul pentru toate cele șase mărimi fizice monitorizate. O problemă majoră în reprezentarea grafică pe dispoz-

itive mobile este densitatea eșantioanelor. Dacă aplicația ar încerca să afișeze fiecare punct salvat pe durata unei săptămâni, ecranul ar deveni supraaglomerat, iar performanța interfeței UI ar scădea. Pentru a soluționa această limitare, algoritmul implementează un sistem dinamic de ajustare a rezoluției în funcție de filtrul temporal selectat de utilizator (o oră, 24 ore sau 7 zile), prezentat și în **Figura 13**, sub formă de schemă logică:

- ▣ **Filtru 1H:** Afișează datele brute la rezoluție maximă, oferind detalii despre modificările imediate.
- ▣ **Filtrele 24H și 7D:** Algoritmul calculează un pas de eșantionare și reține doar al N-lea punct din matricea de date, garantând că numărul maxim de puncte afișate nu depășește constantele de siguranță prestabilite (`MAX_CHART_POINTS`).

Pentru a permite citirea valorilor exacte direct de pe grafic, s-a dezvoltat un sistem de ferestre pop-up (Tooltips) declanșat la atingerea oricărui punct. Deoarece bibliotecile standard oferă un suport limitat pentru interfețe personalizate, ferestrele de dialog sunt construite matematic prin intermediul elementelor scalabile vectoriale, oferite de biblioteca `react-native-svg`. Algoritmul calculează coordonatele spațiale ale punctului atins și afișează dinamic un dreptunghi de contrast.

Dacă utilizatorul dorește să realizeze o analiză detaliată a evoluției microclimatului fermei, arhitectura ecranului include un modul pentru extragerea datelor în format CSV. Prin interacțiunea cu un buton, sistemul apelează funcția personalizată `useExportCSV`. Această funcție extrage cele mai recente 1000 de înregistrări și le transpune într-un fișier de tip CSV. Pentru a garanta uniformitatea informațiilor extrase, algoritmul de parsare verifică starea globală a aplicației înainte de generarea fișierului, aplicând automat conversia matematică a temperaturilor în grade Celsius sau Fahrenheit, în concordanță cu preferințele setate de utilizator.

### 3.5.4 Configurarea de la distanță a sistemului și gestionarea preferințelor utilizatorului

Utilitatea unui sistem IoT complet se bazează nu doar de monitorizarea parametrilor de mediu, ci și de capacitatea de control de la distanță asupra parametrilor fizici. Pentru a elimina necesitatea conectării fizice la unitatea Raspberry Pi pentru modificarea pragurilor de funcționare, aplicația mobilă dispune de un panou avansat de control în ecranul de setări al aplicației.

Interfața de configurare pune la dispoziția cultivatorului 12 câmpuri numerice, corespunzătoare limitelor inferioare și superioare pentru cele 6 mărimi fizice monitorizate. Pentru a asigura consistența datelor înainte de transmiterea lor prin rețea, aplicația validează și scrie valorile introduse de utilizator într-un obiect JSON, respectând structura ierarhică așteptată de baza de date. La inițierea

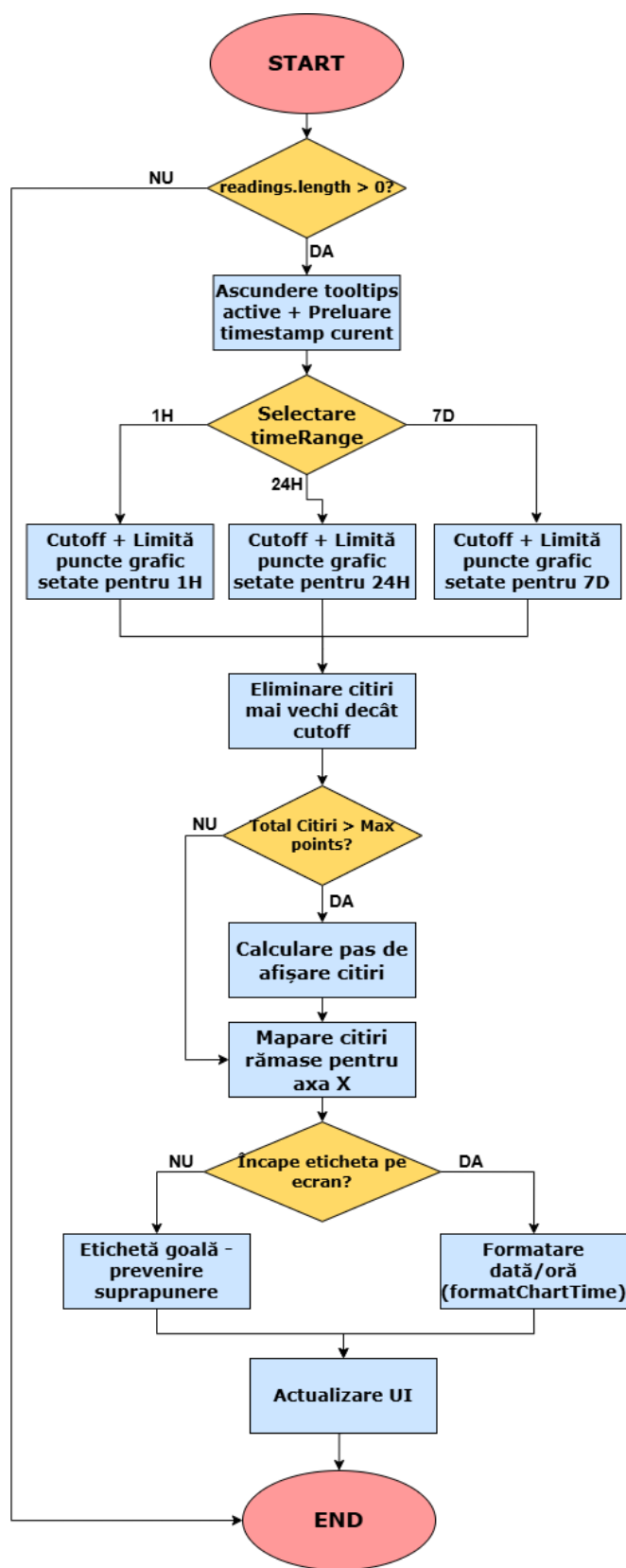


Figura 13: Schema logică a algoritmului de reducere și generare controlată a etichetelor axei X

salvării, aplicația apelează metoda `updateFarmSettings` din stratul de servicii. Această acțiune declanșează suprascrierea nodului `farm_settings` din Firebase. Modificarea nodului din Cloud este interceptată de ascultătorul de evenimente (`_settings_listener`) care rulează în fundal pe Raspberry Pi, actualizând algoritmul de histerezis și bucla de control a ventilatorului în timp real, fără a necesita repornirea întregului sistem fizic.

Pe lângă controlul parametrilor de mediu, ecranul de setări gestionează starea interfeței vizuale și conversia unităților termice. Prin intermediul elementelor de control de tip "switch", utilizatorul poate interacționa cu două stări globale majore:

- **Tema vizuală:** Modificarea acestei variabile de stare este preluată de toate componentele vizuale, actualizând culorile pentru a optimiza vizibilitatea și consumul energetic al ecranului telefonului.
- **Conversia unităților termice (Celsius/Fahrenheit):** Pornirea acestei opțiuni nu alterează datele stocate în baza de date. În schimb, aplică o formulă matematică direct în interfața grafică, recalculând dinamic valorile afișate pe ecranul principal și în graficele istorice.

### 3.5.5 Sistemul de monitorizare vizuală în timp real

O componentă necesară pentru validarea stării biologice a culturii din cadrul fermei o reprezintă modulul de monitorizare vizuală. Din punct de vedere arhitectural, această funcționalitate este izolată într-un ecran dedicat al aplicației mobile, implementat în fișierul `live_camera.tsx`. Totodată, ecranul poate să fie accesat direct din ecranul principal al aplicației, printr-o acțiune de navigare declanșată de utilizator.

În faza de dezvoltare a interfeței grafice în React Native, s-a identificat o provocare legată de comportamentul componentei de afișare a imaginilor. În mod normal, pentru a optimiza consumul de date mobile și performanța afișării, framework-ul implementează un algoritm de stocare în memoria cache a resurselor multimedia preluate prin rețea de la un URL la distanță. Întrucât serverul local asincron expune imaginea curentă prin intermediul unui endpoint public Ngrok cu o adresă statică unică, componenta de tip frontend interpretează resursa ca fiind neschimbată. Ca urmare, aplicația tinde să afișeze în mod repetat prima imagine stocată în memoria cache a smartphone-ului, ignorând cadrele noi actualizate de pe placa Raspberry Pi 5, chiar dacă fișierul fizic de pe unitate se modifică la fiecare 5 secunde.

Pentru a forța componenta grafică să evite utilizarea memoriei cache locale și să efectueze o cerere HTTP completă de fiecare dată când intervalul de eșantionare a trecut, s-a implementat tehnica numită **cache-busting**, care presupune modificarea dinamică a șirului de caractere ce definește adresa

URL, adăugând o componentă unică la finalul acesteia sub forma unui parametru fictiv de interogare. Logica este controlată utilizând hook-urile `useEffect` și `useState`. La deschiderea ecranului `live_camera.tsx`, un cronometru asincron este pornit și configurat să ruleze periodic la un interval de 5 secunde, în simetrie cu rata de captură a camerei USB.

Din perspectiva motorului de randare React Native, modificarea string-ului proprietății sursă a componentei `Image` este percepută ca o schimbare de stare ce indică o adresă URL complet nouă. Astfel, sistemul de operare de pe dispozitivul mobil invalidează cache-ul asociat anterior, declanșează o cerere de tip **HTTPS GET** prin Ngrok, preia din memoria RAM a plăcii Raspberry Pi 5 cea mai recentă poză stocată în `/tmp` și actualizează interfața grafică aproape instant.

### 3.5.6 Sinteza navigării și experiența utilizatorului

Pentru a oferi o perspectivă asupra interacțiunii umane cu sistemul software mobil, interfața a fost modelată pe baza unui flux decizional clar. **Figura 14** ilustrează schema logică a navigării, prezentând traseul utilizatorului de la deschiderea aplicației până la accesarea funcționalităților finale, evidențiind mecanismele de restricționare a accesului.

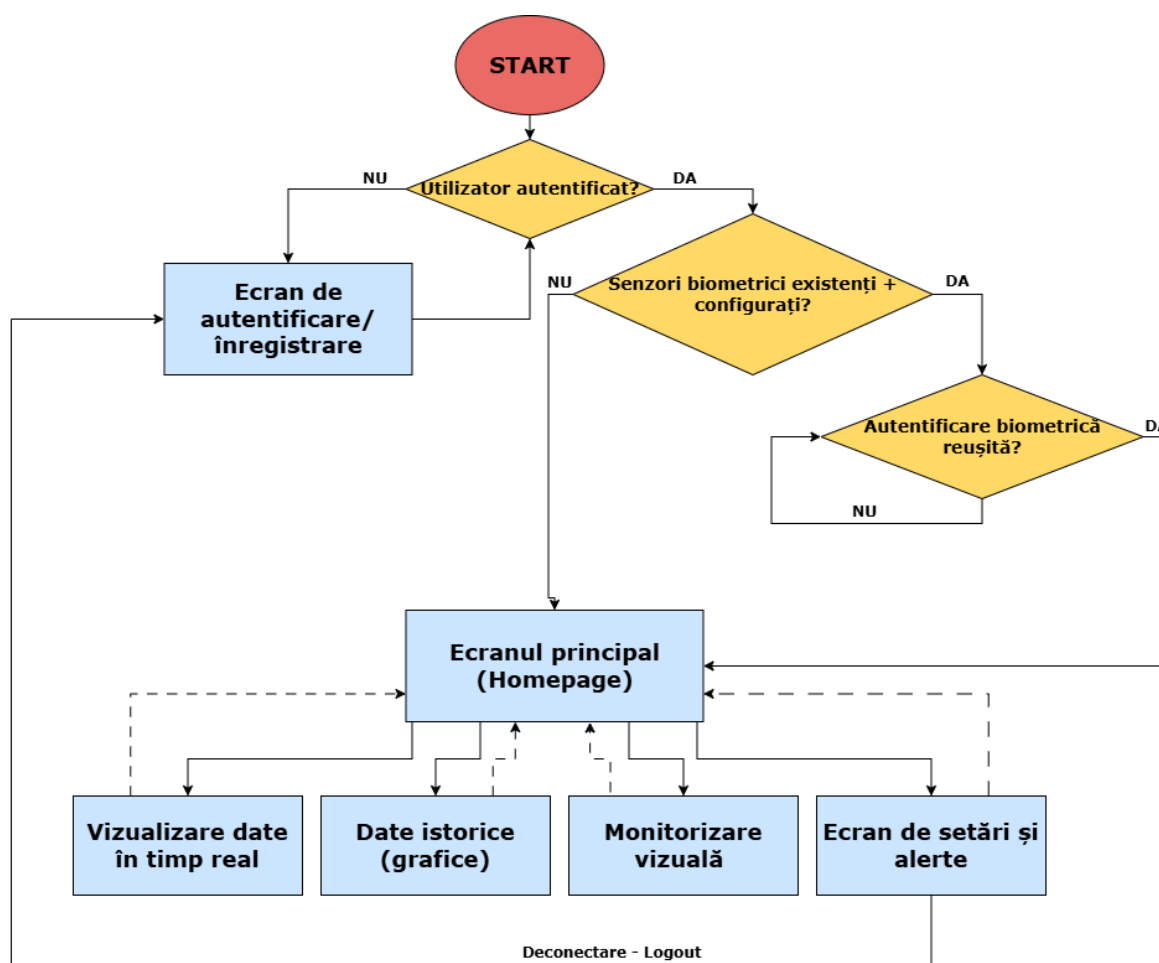


Figura 14: Schema logică a fluxului de navigare a utilizatorului - User Flowchart

Procesul începe cu un sistem de securitate structurat pe două niveluri independente. La inițializarea aplicației, sistemul interoghează asincron starea sesiunii validate prin platforma Firebase (**Figura 15**). În absența unui token activ de autentificare, rutarea blochează accesul la datele fermei și direcționează utilizatorul exclusiv către interfața de înregistrare și logare. Odată confirmată identitatea la nivel de server, fluxul de execuție evaluează capabilitățile hardware ale telefonului gazdă. Dacă terminalul are senzori biometrici configurați în sistemul de operare, aplicația impune o a doua barieră de validare, strict locală, prevenind astfel accesul fizic neautorizat la panoul de comandă al aplicației, în cazul în care telefonul este deblocat de o persoană neautorizată.

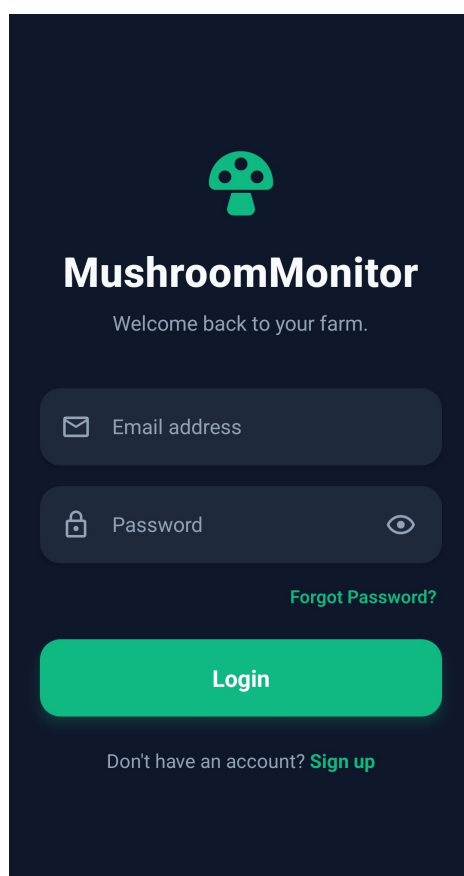
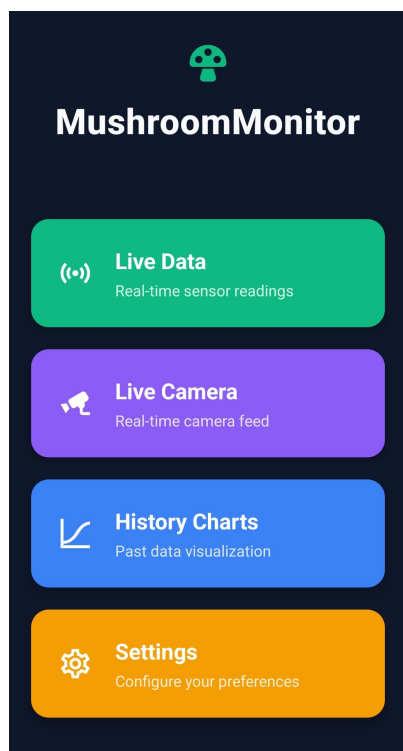


Figura 15: Ecranul de autentificare

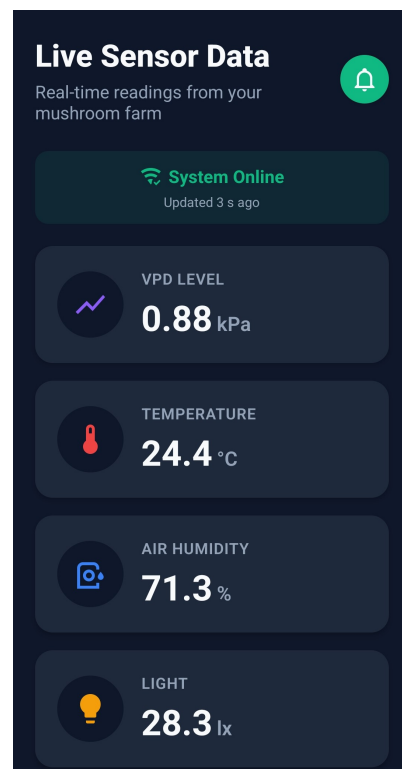
Depășirea cu succes a protocoalelor de securitate conduce utilizatorul în nodul central al aplicației, mai exact ecranul de tip **Homepage**. Acesta oferă utilizatorului patru rute diferite către modulele funcționale majore (**Figura 16a și Figura 16b**) ale sistemului IoT: vizualizarea parametrilor în timp real, analiza grafică a datelor istorice, monitorizarea vizuală prin intermediul sistemului de captură asincronă și configurarea pragurilor de alertare.

Modulele de analiză grafică și configurare permit gestionarea continuă a microclimatului. Utilizatorul poate vizualiza istoricul datelor stocate în Cloud sau poate modifica dinamic pragurile de alertă, noile valori fiind aplicate instant (**Figura 16c și Figura 16d**).

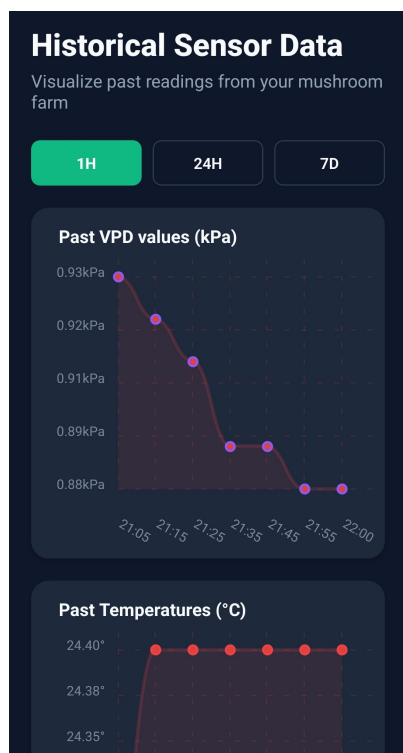
Ciclul de utilizare al aplicației se poate încheia în mod controlat prin declanșarea acțiunii de de-



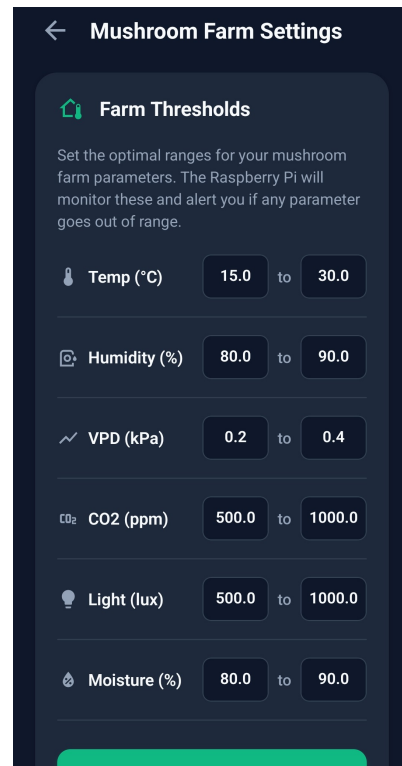
a) Ecran principal



b) Monitorizare parametri în timp real



c) Analiză grafică istorică



d) Configurarea pragurilor de alertă

Figura 16: Interfața aplicației: modulul principal, monitorizarea, analiza istorică și configurarea



conectare din meniul de setări. Această operațiune distruge tokenul de acces curent și redirecționează forțat aplicația înapoi la starea inițială, asigurând închiderea completă a buclei de navigare și protejarea sistemului.

## 4 Concluzii

### 4.1 Îndeplinirea obiectivelor și rezultate obținute

Scopul principal al proiectului de diplomă a fost atins prin proiectarea, implementarea și validarea unui sistem hibrid complet (hardware-software) pentru monitorizarea și controlul automat al unei ferme de ciuperci. Rezultatul final reprezintă un prototip funcțional ce integrează concepte de Edge Computing, capabil să evalueze și să stabilizeze factorii de mediu independenți de latențele unei rețele externe.

Obiectivele specifice gândite în faza de proiectare au fost îndeplinite cu succes. Ansamblul hardware a fost izolat fizic pentru a rezista într-un mediu cu umiditate ridicată de 80-90%, iar citirea datelor a fost realizată cu precizie prin combinarea protocolelor seriale I<sup>2</sup>C și 1-Wire cu un convertor analog-digital extern ADS1115. De asemenea, fluxul multimedia asincron furnizat de camera web USB a fost expus cu succes în exterior prin tehnologia de tunelare Ngrok, oferind utilizatorului o perspectivă reală asupra culturii.

### 4.2 Contribuții personale

Dezvoltarea acestui sistem a presupus o abordare interdisciplinară, impunând cercetare și implementare pe multiple straturi tehnologice. Principalele contribuții personale aduse în cadrul acestui proiect sunt:

- **Dezvoltarea driverelor software de la zero:** Implementarea claselor orientate pe obiecte în Python pentru interpretarea datelor brute de la senzori, incluzând prelucrarea matematică a ecuației Tetens pentru calculul VPD-ului și extrapolarea logaritmică a nivelului de CO<sub>2</sub> (PPM).
- **Arhitectura software concurentă:** Proiectarea sistemului pe trei fire de execuție independente (monitorizare hardware, comunicație Cloud, streaming multimedia) sincronizate prin structuri de tip coadă (queue), eliminând riscul de tip *Single Point of Failure*.
- **Implementarea mecanismului de invalidare a cache-ului și a stocării în memoria RAM:** Proiectarea tehnicii de stocare a cadrelor foto direct în partiția /tmp a sistemului Linux, cuplată cu logica reactivă din aplicația mobilă pentru forțarea descărcării noilor imagini de la URL-ul Ngrok.
- **Proiectarea arhitecturii Zero-Trust în aplicația mobilă:** Implementarea securității biometrice la nivel de dispozitiv și a rutării bazate pe starea sesiunii Firebase.

### 4.3 Evaluarea metricilor de performanță

Pentru a valida ingineria sistemului propus, s-a realizat o evaluare a deciziilor arhitecturale prin trei metrici de performanță esențiale:

1. **Eficiența optimizării stocării:** Sistemul de monitorizare vizuală extrage un cadru la un interval de 5 secunde. Într-un ciclu de 24 de ore, acest proces generează 17280 de fișiere de imagine de tip JPG. Salvarea acestora direct pe memoria Flash ar fi provocat epuizarea rapidă a ciclurilor de citire/scriere. Prin direcționarea fluxului către memoria RAM, rata de folosire a cardului a fost redusă la zero, garantând sustenabilitatea prototipului pe termen lung.

```

C:\. andi@raspberrypi: ~
File: /tmp/live_view.jpg
Size: 400019      Blocks: 784      IO Block: 4096   regular file
DeSize: 399776   Inode: 2      Blocks: 784     Links: 1      IO Block: 4096   regular file
Device: 179,2    Inode: 23006  id: ( 1      Links: 1      andi)   Gid: ( 1000/   andi)
Access: (0644/-rw-r--r--)  Uid: ( 1000/   andi)   Gid: ( 1000/   andi)
Access: 2026-06-16 16:02:59.172010932 +0300
Modify: 2026-06-16 17:13:26.848551935 +0300
Change: 2026-06-16 17:13:26.848551935 +0300
Birth: 2026-06-16 16:02:59.172010932 +0300

C:\. andi@raspberrypi: ~
Total DISK READ:      0.00 B/s | Total DISK WRITE:      63.32 K/s
Current DISK READ:    0.00 B/s | Current DISK WRITE:    411.60 K/s
  TID  PRIO  USER      DISK READ  DISK WRITE  COMMAND
  ----  ---  -
  265  be/3  root      0.00 B/s   63.32 K/s  [jbd2/mmcblk0p2-8]

```

Figura 17: Demonstrarea separării fluxului de scriere pe microSD prin monitorizarea memoriei RAM și a proceselor de scriere pe disc

În **Figura 17** se observă faptul că, deși sistemul captează și stochează asincron fotografii cu o rezoluție înaltă, fluxul de date ocolește controller-ul de memorie flash al cardului microSD. Pentru a demonstra acest aspect, s-a realizat o verificare la nivel de procese folosind utilitarul `iotop`. În același timp, au fost urmăriți timpii de modificare a fișierului `/tmp/live_view.jpg` din memoria RAM. Se poate observa că, în momentul generării și suprascrierii fișierului imagine, singurele operațiuni fizice de scriere pe discul `/dev/mmcblk0` sunt generate de procesul `jbd2` (Journaling Block Device 2). Absența procesului aplicației `python3` din stiva operațiunilor de scriere pe disc validează decizia de a utiliza partiția `/tmp`. Această optimizare protejează cardul de memorie și oferă un număr infinit de cicluri de stocare multimedia.

2. **Evaluarea gradului de încărcare hardware și a stabilității resurselor:** Sistemul a fost evaluat din punct de vedere al consumului de resurse pe unitatea centrală Raspberry Pi 5 pentru

a justifica utilizarea unei arhitecturi Single-Board Computer în locul unui microcontroller clasic. În timpul rulării concurente a celor trei fire de execuție, monitorizarea proceselor la nivelul sistemului de operare Linux a evidențiat următoarele aspecte:

- **Stabilitatea memoriei RAM:** Pentru a demonstra eficiența gestionării resurselor și absența scurgerilor de memorie, s-a monitorizat continuu spațiul de memorie fizică reținut de procesele principale, folosind comanda de sistem `ps` cu urmărirea parametrului RSS. În **Figura 18** se observă că scriptul principal al aplicației, care gestionează eșantionarea

```
Every 2.0s: ps -C python3 -o pid,%cpu,%mem,rss,command          raspberrypi: Tue Jun 16 17:29:42 2026
  PID %CPU %MEM    RSS COMMAND
   816   0.0   0.2 18992 /home/andi/proiect_licenta/project_env/bin/python3 /home/andi/proiect_licenta/image_server.py
 14120   0.8   0.7 63040 /home/andi/proiect_licenta/project_env/bin/python3 /home/andi/proiect_licenta/proiect_licenta.py
```

Figura 18: Monitorizarea consumului de resurse pentru procesele concurente ale sistemului local

senzorilor și comunicarea cu platforma Cloud, menține un spațiu de aproximativ 61 MB, reprezentând doar 0.7% din memoria totală a sistemului. În paralel, serverul asincron de imagini ocupă aproximativ 18.5 MB. Datorită implementării cozilor de mesaje cu dimensiune de 10 elemente și a eliberării constante a referințelor, aceste valori rămân constante pe perioade lungi de funcționare.

- **Toleranța interfeței de rețea și integritatea datelor:** Capacitatea modulului intern Wi-Fi de a susține trafic a fost validată prin analiza statisticilor de transmisie la nivelul interfeței `wlan0`, utilizând comanda de sistem `ifconfig`. Așa cum este prezentat în **Figura**

```
andi@raspberrypi:~ $ ifconfig wlan0
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.185.213 netmask 255.255.255.0 broadcast 192.168.185.255
    inet6 fe80::42b9:ee:ed20:e159 prefixlen 64 scopeid 0x20<link>
    ether 2c:cf:67:13:b5:ab txqueuelen 1000 (Ethernet)
    RX packets 39704 bytes 18893886 (18.0 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 45117 bytes 26020230 (24.8 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figura 19: Statisticile de trafic pentru interfața wireless wlan0

**19**, în urma unui trafic bidirecțional susținut de monitorizare și captură foto, însumând peste 40 MB transferați, parametrii asociați erorilor de rețea au rămas la valoarea zero. Contoarele hardware pentru erorile de recepție/transmisie (RX/TX errors), precum și cele pentru pachetele aruncate (dropped), raportează lipsa oricăror incidente. Acest rezultat experimental confirmă faptul că arhitectura hardware menține simultan conexiunea per-

sistentă de tip WebSocket cu platforma Firebase și fluxul HTTP asincron necesar tunelului Ngrok, asigurând o comunicare stabilă și eliminând complet fenomenul de packet loss.

- Disponibilitatea resurselor de calcul și comportamentul termic: Pentru a valida rezervele de calcul și eficiența managementului termic, s-a verificat o situație în care sistemul de răcire este activat. Comportamentul hardware a fost monitorizat utilizând `htop` pentru încărcarea procesorului și comanda hardware `vcgencmd` pentru senzorul termic. În **Figura 20** este prezentat atât gradul de utilizare a celor patru nuclee ale procesorului, cât și temperatura unității centrale în timpul funcționării sistemului de răcire activă. Astfel, chiar dacă execuția concurentă a proceselor de achiziție, a tunelului Ngrok și a serverului asincron de imagini rulează împreună, gradul de utilizare a celor patru nuclee se menține la un nivel neglijabil (sub 2-3% per nucleu). Această valoare confirmă faptul că Raspberry Pi 5 poate susține fără probleme viitoare integrări computaționale intensive (precum algoritmi de Machine Learning sau procesare video), fără a necesita actualizări hardware. În ceea ce privește regimul termic, temperatura internă a sistemului pe cip s-a stabilizat la valori mult sub pragul de thermal throttling specific acestor microcalculatoare. Acest rezultat validează eficiența sistemului de răcire activă în disiparea energiei termice generate în regim de funcționare, asigurând astfel stabilitatea nodului Edge și prevenind eventuale scăderi de performanță ale aplicației.

#### 4.4 Analiza tehnico-economică și evaluarea soluțiilor de pe piață

Sistemul dezvoltat a fost conceput având ca grup țintă micii producători și antreprenorii din agricultura urbană. Din punct de vedere economic, costul de producție al prototipului a fost menținut la un nivel minim prin utilizarea componentelor de tip open-source hardware și dezvoltarea software-ului de la zero, evitând astfel licențele comerciale.

Pentru a demonstra fezabilitatea economică a proiectului, **Tabelul 1** prezintă costul estimativ al componentelor utilizate pentru construcția nodului central și a rețelei de senzori. Prețurile afișate au fost preluate din momentul achiziționării componentelor aferente.

Așa cum rezultă din analiza costurilor estimative, investiția hardware necesară se situează în jurul valorii de 150 de euro. Acest cost este extrem de competitiv raportat la funcționalitățile complexe oferite, depășind opțiunile disponibile pe piață. Comparând produsele comerciale existente pe piață la momentul actual, putem observa faptul că:

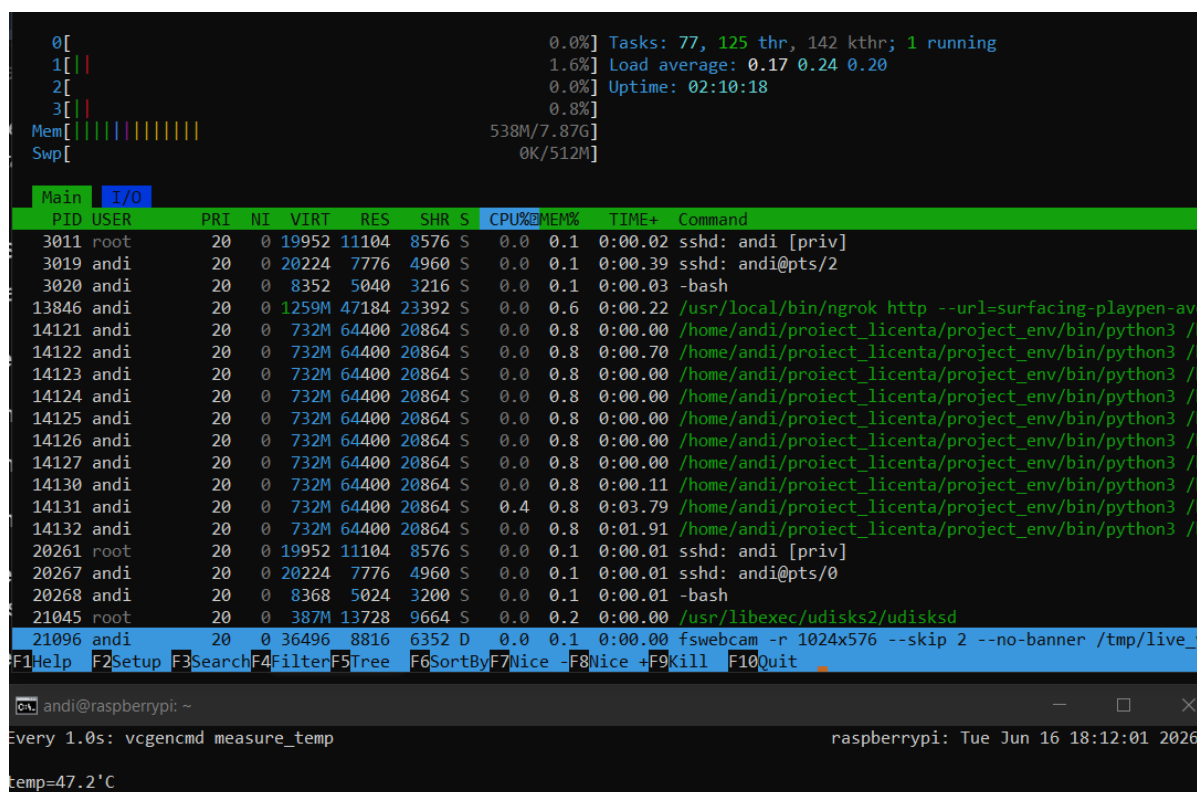


Figura 20: Monitorizarea resurselor de calcul și a temperaturii interne a unității centrale în timpul funcționării sistemului de răcire activă

| Nr.          | Componentă hardware                                                                         | Rol în sistem                                          | Cost (EUR)    |
|--------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------|---------------|
| 1            | Raspberry Pi 5 (8GB) + Alimentator 27W + Card microSD                                       | Unitate centrală                                       | 100.00        |
| 2            | Active Cooler (Raspberry Pi)                                                                | Element de execuție                                    | 6.00          |
| 3            | Camera Web USB                                                                              | Monitorizare vizuală                                   | 12.00         |
| 4            | ADS1115 (Convertor ADC pe 16 biți)                                                          | Digitizarea semnalelor analogice                       | 3.50          |
| 5            | Ansamblu senzori digitali (DS18B20, AM2302, BH1750)                                         | Măsurare temperatură, umiditate aer, lumină            | 6.40          |
| 6            | Ansamblu senzori analogici (MQ-135, HW-101)                                                 | Măsurare concentrație CO <sub>2</sub> și umiditate sol | 4.50          |
| 7            | Ecran OLED SSD1306 (0.96 inch)                                                              | Interfață vizuală locală                               | 3.80          |
| 8            | Materiale auxiliare (placă perforată, ribbon pentru conectare, fire de cupru izolate, pini) | Interconectare placă perforată - Raspberry Pi 5        | 14.00         |
| <b>Total</b> |                                                                                             |                                                        | <b>150.20</b> |

Tabel 1: Costul estimativ al componentelor hardware utilizate în dezvoltarea prototipului

- Dispozitivele independente de tip **Inkbird** prezintă un cost de aproximativ 50-100 euro, însă funcționează asemenea unor controllere separate [4], având capacitatea de a monitoriza un singur parametru. Nu oferă o arhitectură Cloud integrată pentru centralizarea a șase parametri simultan și nu dispun de capacități de calcul local sau monitorizare vizuală.
- Soluțiile integrate de tip **Shrooly** costă aproximativ 300 de euro [3], însă limitează cultivarea ciupercilor la dimensiunea fixă a aparatului, oferind un dezavantaj când vine vorba de scalabilitatea produsului. Totodată, soluția comercială menționată utilizează conectivitatea Bluetooth pentru interfațarea cu dispozitivul, limitând raza de control a utilizatorului asupra fermei. În contrast, arhitectura propusă în această lucrare utilizează conectivitatea la Internet prin intermediul platformei Cloud, permițând monitorizarea și controlul parametrilor de mediu de la distanță, eliminând constrângerile spațiale specifice comunicării de scurtă distanță.
- Echipamentele modulare de tip **TrolMaster**, deși extrem de precise, necesită un buget de mii de euro pentru modulul central și rețeaua de senzori periferici [5], fiind inaccesibile segmentului de piață la care se adresează prototipul construit.

Astfel, sistemul prezentat reprezintă o soluție de mijloc cu un raport cost-performanță bun, comparativ cu soluțiile prezentate anterior. Prototipul elimină limitările de volum printr-o arhitectură modulară detașată și oferă instrumente de analiză avansate la un preț mai mic față de sistemele de automatizare industriale.

## 4.5 Direcții viitoare de cercetare și dezvoltare

Datorită arhitecturii flexibile și orientate pe obiecte, produsul software și hardware reprezintă o fundație solidă pentru implementări viitoare. Extinderea funcționalităților prototipului se îndreaptă spre următoarele direcții:

- Integrarea unor actuatori de putere mare, prin implementarea unor echipamente de control performante, conectate prin intermediul unor relee electromagnetice la pinii liberi ai convertorului ADC sau ai plăcii de dezvoltare.
- Implementarea unor algoritmi de Computer Vision, utilizând fluxul constant de imagini de la camera USB, pentru a detecta automat stadiul optim de recoltare al corpurilor de fructificare sau apariția unor mucegaiuri contaminante, procese care în prezent necesită acțiune umană.
- Implementarea unui modul de alimentare neîntreruptibilă de tip UPS și alimentarea din panouri solare cu acumulatori, anulând astfel vulnerabilitatea blocării buclei de control locale în cazul

unor pene de curent.



# Bibliografie

- [1] *Co2 levels critical to mushroom farm success*, <https://www.co2meter.com/blogs/news/co2-mushroom-farming>, Accesat: 28.05.2026.
- [2] *How to maintain humidity in mushroom cultivation*, <https://atlas-scientific.com/blog/how-to-maintain-humidity-in-mushroom-cultivation/?srsltid=AfmBOoohx4WufiE6H7ww0PyuGmHWVfiBM8YSI7ZEtE>, Accesat: 28.05.2026.
- [3] *Shrooly the at-home mushroom growing system*, <https://shrooly.com/pages/product>, Accesat: 03.06.2026.
- [4] *Inkbird user manuals*, <https://www.inkbird.com/pages/download>, Accesat: 03.06.2026.
- [5] *Trolmaster*, <https://www.trolmaster.com/Home/Index>, Accesat: 03.06.2026.
- [6] R. Gong, H. Zhang, G. Li, and J. He, "Edge computing-enabled smart agriculture: Technical architectures, practical evolution, and bottleneck breakthroughs," *Sensors*, vol. 25, p. 5302, 2025. DOI: 10.3390/s25175302 [Online]. Available: <https://doi.org/10.3390/s25175302>
- [7] P. Yu, F. Teng, W. Zhu, C. Shen, Z. Chen, and J. Song, "Cloud-edge-device collaborative computing in smart agriculture: Architectures, applications, and future perspectives," *Frontiers in Plant Science*, vol. 16, p. 1668545, 2025. DOI: 10.3389/fpls.2025.1668545 [Online]. Available: <https://doi.org/10.3389/fpls.2025.1668545>
- [8] S.-T. Chang and P. G. Miles, *Mushrooms: Cultivation, Nutritional Value, Medicinal Effect, and Environmental Impact*, 2nd. CRC Press, 2004, ISBN: 0-8493-1043-1.
- [9] B. Sood Phd and M. Pecht, "Controlling moisture in printed circuit boards," *IPC APEX EXPO Technical Conference 2011*, vol. 1, Jan. 2011.
- [10] W. Wolf, *Computers as Components: Principles of Embedded Computing System Design*, 3rd. Morgan Kaufmann, 2012, ISBN: 978-0-12-374397-8.
- [11] *Um10204: I2c-bus specification and user manual*, Accesat: 30.05.2026, NXP Semiconductors, 2021. [Online]. Available: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>

- [12] G. Carmen, *Interfețe seriale - i2c*, Lecture presented at Transilvania University of Brașov, Protocole și interfețe, 2021.
- [13] *Overview of 1-wire technology and its use*, <https://www.analog.com/en/resources/technical-articles/guide-to-1wire-communication.html>, Accesat: 30.05.2026.
- [14] Maxim Integrated Products, *Ds18b20 datasheet*, Accesat: 03.06.2026, 2019. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>
- [15] *Introduction to nosql*, <https://www.geeksforgeeks.org/dbms/introduction-to-nosql/>, Accesat: 31.05.2026.
- [16] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017, ISBN: 978-1449373320.
- [17] V. Wang, F. Salim, and P. Moskovits, *The Definitive Guide to HTML5 WebSocket*. Apress, 2013, ISBN: 978-1430247401.
- [18] Google, *Firebase realtime database documentation*, Accesat: 31.05.2026, Google LLC, 2026. [Online]. Available: <https://firebase.google.com/docs/database>
- [19] N. Hazdun, *How to create a websocket application: A step-by-step guide*, <https://geniusee.com/single-blog/how-to-build-a-websocket-application>, Accesat: 01.06.2026.
- [20] *Pub/sub architecture*, <https://www.geeksforgeeks.org/system-design/what-is-pub-sub/>, Accesat: 01.06.2026.
- [21] Google, *Firebase realtime database documentation: Offline capabilities*, Accesat: 01.06.2026, Google LLC, 2026. [Online]. Available: <https://firebase.google.com/docs/database/android/offline-capabilities>
- [22] B. Eisenman, *Learning React Native: Building Native Mobile Apps with JavaScript*. O'Reilly Media, 2015, ISBN: 978-1491929001.
- [23] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," National Institute of Standards and Technology (NIST), Special Publication 800-207, 2020. DOI: 10.6028/NIST.SP.800-207
- [24] OWASP Foundation, *Mobile application security verification standard (masvs)*, Accesat: 01.06.2026, 2024. [Online]. Available: <https://mas.owasp.org/MASVS/>

- [25] J. Ariza and H. Baez, "Understanding the role of single-board computers in engineering and computer science education: A systematic literature review," *Computer Applications in Engineering Education*, vol. 30, no. 1, pp. 304–329, 2022. DOI: <https://doi.org/10.1002/cae.22439> [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cae.22439>
- [26] E. Upton and G. Halfacree, *Raspberry Pi User Guide*, 4th. Wiley, 2016, ISBN: 978-1119264361.
- [27] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, pp. 1–1, Oct. 2016. DOI: 10.1109/JIOT.2016.2579198
- [28] P. Marwedel, *Embedded System Design: Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things*, 4th. Springer, 2021, ISBN: 978-3030606973.
- [29] *What is python?* <https://aws.amazon.com/what-is/python/>, Accesat: 01.06.2026.
- [30] M. Lutz, *Learning Python*, 5th. O'Reilly Media, 2013, ISBN: 978-1449355739.
- [31] *Python documentation: The python standard library*, Accesat: 02.06.2026, Python Software Foundation, 2026. [Online]. Available: <https://docs.python.org/3.16/library/index.html#the-python-standard-library>
- [32] *Python documentation: Asyncio - asynchronous i/o*, Accesat: 02.06.2026, Python Software Foundation, 2026. [Online]. Available: <https://docs.python.org/3/library/asyncio.html#module-asyncio>
- [33] T. Siddiqui, *Introduction to typescript and it's features*, <https://www.htmlgoodies.com/javascript/introduction-to-typescript-and-its-features/>, Accesat: 02.06.2026.
- [34] B. Cherny, *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media, 2019, ISBN: 978-1492037651.
- [35] *Apache cordova documentation*, Accesat: 02.06.2026, The Apache Software Foundation, 2026. [Online]. Available: <https://cordova.apache.org/docs/en/latest/guide/overview/>
- [36] Expo Team, *Expo documentation: Faq*, Accesat: 02.06.2026, 2026. [Online]. Available: <https://docs.expo.dev/faq/>
- [37] I. Igboabu, *What is backend as a service (baas)? a beginner's guide*, <https://www.freecodecamp.org/news/backend-as-a-service-beginners-guide>, Accesat: 02.06.2026.
- [38] L. Moroney, *The Definitive Guide to Firebase: Build Android Apps on Google's Mobile Platform*. Apress, 2017, ISBN: 978-1484229439.

- [39] Google, *Understand firebase realtime database security rules*, Accesat: 02.06.2026, Google LLC, 2026. [Online]. Available: <https://firebase.google.com/docs/database/security>
- [40] P. R. Pukale, K. Sri Rangam, S. P. Jitendra, D. Lee, and J. Gao, "An intelligent iot navigation system for visually impaired individuals: Design and implementation," in *2025 IEEE World AI IoT Congress (AllIoT)*, 2025, pp. 0787–0794. DOI: 10.1109/AIIoT65859.2025.11105368
- [41] *Ngrok documentation - what is ngrok?* Accesat: 12.06.2026.
- [42] *Ngrok documentation - secure tunnels overview*, Accesat: 12.06.2026.
- [43] *Introduction to github*, <https://www.geeksforgeeks.org/git/introduction-to-github/>, Accesat: 03.06.2026.
- [44] Raspberry Pi Ltd, *Raspberry pi 5 datasheet*, Accesat: 03.06.2026, 2026. [Online]. Available: <https://pip-assets.raspberrypi.com/categories/892-raspberry-pi-5/documents/RP-008348-DS-4-raspberry-pi-5-product-brief.pdf?disposition=inline>
- [45] A. Singh, *Raspberrypi 5 pinout, specifications, pricing a complete guide*, <https://hackatronic.com/raspberry-pi-5-pinout-specifications-pricing-a-complete-guide/>, Accesat: 03.06.2026.
- [46] Texas Instruments, *Ads111x datasheet*, Accesat: 03.06.2026, 2024. [Online]. Available: <https://www.ti.com/lit/ds/symlink/ads1115.pdf>
- [47] *Ads1115 cjmcu analog-digital converter module (adc)*, [https://www.optimusdigital.ro/en/analog-to-digital-converters-adcs/1696-ads1115-cjmcu-digital-analogic-converter-module-adc.html?srsltid=AfmB0ooWrDB5Sl\\_7NooHDVfneBZt8eAa2mw7ZuIsAkoSHsjV](https://www.optimusdigital.ro/en/analog-to-digital-converters-adcs/1696-ads1115-cjmcu-digital-analogic-converter-module-adc.html?srsltid=AfmB0ooWrDB5Sl_7NooHDVfneBZt8eAa2mw7ZuIsAkoSHsjV) Accesat: 03.06.2026.
- [48] Aosong Electronics, *Am2302 datasheet*, Accesat: 04.06.2026. [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/Digital+humidity+and+temperature+sensor+AM2302.pdf>
- [49] Mouser Electronics, *Bh1750 datasheet*, Accesat: 04.06.2026. [Online]. Available: [https://www.mouser.com/datasheet/2/348/bh1750fvi-e-186247.pdf?srsltid=AfmB0oqoS4sMoej80y5pJX7bFP5IZwF\\_BiUe](https://www.mouser.com/datasheet/2/348/bh1750fvi-e-186247.pdf?srsltid=AfmB0oqoS4sMoej80y5pJX7bFP5IZwF_BiUe)
- [50] Zhengzhou Winsen Electronics Technology, *Mq-135 datasheet*, Accesat: 04.06.2026. [Online]. Available: [https://www.winsen-sensor.com/d/files/PDF/Semiconductor%20Gas%20Sensor/MQ135%20\(Ver1.4\)%20-%20Manual.pdf](https://www.winsen-sensor.com/d/files/PDF/Semiconductor%20Gas%20Sensor/MQ135%20(Ver1.4)%20-%20Manual.pdf)

- [51] J. Fuller, *Fc-28 soil moisture sensor module*, <https://www.datasheethub.com/fc-28-soil-moisture-sensor-module/>, Accesat: 04.06.2026.
- [52] Shenzhen Haiwang Sensor, *Hw-101 datasheet*, Accesat: 04.06.2026. [Online]. Available: <https://www.datocms-assets.com/28969/1662716326-hw-101-hw-moisture-sensor-v1-0.pdf>
- [53] *Lcd display vs oled: Which is better and why*, [https://riverdi.com/blog/lcd-display-vs-oled-which-is-better-and-why?srsltid=AfmB0oqMX5QRixd9b8H4UpP\\_-Wp3ptQuCaNXN9tcvc8D3fBje4](https://riverdi.com/blog/lcd-display-vs-oled-which-is-better-and-why?srsltid=AfmB0oqMX5QRixd9b8H4UpP_-Wp3ptQuCaNXN9tcvc8D3fBje4), Accesat: 04.06.2026.
- [54] *Ssd1306 oled display*, <https://esphome.io/components/display/ssd1306/>, Accesat: 04.06.2026.
- [55] D. Gironi, *Cheap co2 meter using the mq135 sensor with avr atmega*, <https://davigeroni.blogspot.com/2014/01/cheap-co2-meter-using-mq135-sensor-with.html>, Accesat: 06.06.2026.
- [56] *Technical data mq-135 gas sensor*, Accesat: 06.06.2026.
- [57] *Tetens equation*, [https://en.wikipedia.org/wiki/Tetens\\_equation](https://en.wikipedia.org/wiki/Tetens_equation), Accesat: 09.06.2026.

## Rezumat

Lucrarea prezintă proiectarea, implementarea și validarea unui sistem hibrid IoT (Edge-Cloud) specializat monitorizării și controlului automatizat al unei ferme de ciuperci. Spre deosebire de culturile vegetale, ciupercile au nevoie de un control amănunțit al temperaturii, umidității și concentrației de CO<sub>2</sub>. Întrucât soluțiile comerciale prezente pe piață sunt fie fragmentate funcțional, fie inaccesibile financiar pentru micii producători, proiectul propune o alternativă modulară și scalabilă, oferind un cost de producție competitiv celorlaltor soluții.

Arhitectura hardware utilizează un microcalculator Raspberry Pi 5, ce are rolul de nod Edge Computing, amplasat în exteriorul mediului pentru a proteja componentele de umiditatea excesivă de 80-90%. Acesta colectează date printr-o rețea de senzori digitali și analogici, utilizând protocoalele I<sup>2</sup>C și 1-Wire, alături de un convertor ADC, cu rol în digitizarea semnalelor analogice.

La nivel software, sistemul rulează local scripturi Python bazate pe rulare concurentă, care asigură prelucrarea matematică a semnalelor. Pentru capturile vizuale, s-a utilizat partiția memoriei RAM, eliminând complet posibilitatea deteriorării memoriei flash a unității centrale.

Infrastructura de rețea folosește platforma Firebase Realtime Database pentru sincronizare bidirecțională prin WebSockets și pe serviciul Ngrok pentru tunelarea securizată a fluxului de capturi. Interacțiunea cu utilizatorul este realizată printr-o aplicație mobilă dezvoltată în React Native și TypeScript. Aceasta integrează un model de securitate Zero-Trust cu autentificare biometrică, permițând monitorizarea în timp real, analiza grafică a datelor istorice cu export CSV, detectarea stării de deconectare și modificarea de la distanță a pragurilor de funcționare.

Evaluarea performanțelor confirmă stabilitatea resurselor computaționale, absența pierderilor de pachete de rețea și o eficiență tehnico-economică superioară celorlaltor produse comerciale existente.

## Abstract

This thesis presents the design, implementation and validation of a hybrid IoT (Edge-Cloud) system dedicated to the automated monitoring and control of a fungal microclimate. Unlike green plants, mushrooms require a detailed management of temperature, humidity and CO<sub>2</sub> levels. Because current commercial solutions are either functionally fragmented or financially inaccessible for small-scale urban farmers, this project proposes a modular, scalable and highly cost-effective alternative.

The hardware architecture relies on a Raspberry Pi 5 acting as an Edge Computing node. To prevent hardware degradation caused by high humidity, the computing unit is isolated outside the cultivation environment. It acquires data from a network of digital and analog sensors using I<sup>2</sup>C and 1-Wire protocols, facilitated by an external analog-to-digital converter.

On the software level, the local system executes concurrent Python scripts responsible for mathematical signal processing. The visual monitoring is optimized by storing captured images directly in the RAM partition, entirely eliminating SD card wear.

The network infrastructure uses Firebase Realtime Database for low-latency bidirectional synchronization, alongside Ngrok for secure photo captures tunneling. User interaction is facilitated by a mobile application developed in React Native and TypeScript. The app integrates a Zero-Trust security model with local biometric authentication, enabling real-time data tracking, historical data visualization with CSV export, offline state detection and remote configuration of critical environmental thresholds.

Performance evaluations confirm high computational resource stability, zero packet loss over the wireless interface and an excellent technical-economic ratio compared to existing commercial automation systems.